Chapter 14 — Interview Question Bank

How to use this bank

This chapter collects the microservices questions that come up again and again in interviews: the ones on popular question lists (InterviewBit, GeeksforGeeks, devinterview.io, DataCamp, TechPrep), in LeetCode Discuss interview-experience posts for backend and Java/Spring roles, and in senior "scenario" rounds. They are worded the way interviewers usually ask them.

- ★ marks a question that appears on many lists or is reported very often. If you are short on time, drill the ★ questions first.
- Every question has a **short answer you can say out loud** in 30–60 seconds. The answers are written for this book and assume the versions used in the rest of it: **Java 25, Spring Boot 4.1, Spring Framework 7, Spring Cloud 2025.1 ("Oakwood"), Kafka, PostgreSQL, Kubernetes**.
- → **More:** links point to the chapter (or the section of Chapter 1) that explains the topic in full. Questions without a link are not covered elsewhere in the book, so the answer here is the complete one.
- New terms are defined the first time they appear. If a word is still unfamiliar, the 300-term glossary at the end of Ch 1 — System Design Fundamentals has it.

How to drill: cover the answer, say your own answer out loud, then uncover and compare. Mark the ones you fumbled and repeat them the next day. Interviewers care less about exact wording than about whether you name the *trade-off* — almost every good answer here ends with "the cost is…" or "use it when…".

---

1. Fundamentals: monolith vs microservices

Q1. ★ What are microservices? Microservices is an architectural style in which an application is built as a set of small services, each owning one business capability, running in its own process, owning its own data and deployable independently. Services talk over the network (HTTP, gRPC or messaging). The point is *independent change*: one team can build, test, deploy and scale its service without coordinating a release with everyone else. → *More:* Ch 1 — Architectural styles

Q2. ★ What is the difference between a monolithic and a microservices architecture? A monolith is one deployable unit: all features share one codebase, one process and usually one database, and calls between modules are in-memory function calls. Microservices split those features into separate deployables that call each other over the network and keep separate databases. A monolith is simpler to build, test, debug and run; microservices give independent deployment, independent scaling and team autonomy, at the price of network failures, distributed data and much more operational work. → *More:* Ch 1 — Architectural styles · Ch 2 — Strangler Fig, Part 0

Q3. ★ What are the advantages of microservices? Independent deployment (ship one service without redeploying everything), independent scaling (scale only the hot service), fault isolation (one crash need not take down the whole product), technology freedom per service, and smaller codebases that a single team can understand and own. The biggest real-world benefit is organisational: many teams can move in parallel with fewer merge conflicts and release trains.

Q4. ★ What are the disadvantages or challenges of microservices? Every call is now a network call that can be slow or fail; data is spread across databases so you lose easy joins and ACID transactions; debugging needs distributed tracing; testing needs contract tests; you need service discovery, config management, CI/CD per service, monitoring and on-call for dozens of things. Versioning APIs between teams and keeping data eventually consistent are ongoing costs. This is often called the "microservices tax". → *More:* Ch 1 — Synchronous vs asynchronous communication

Q5. ★ When should you NOT use microservices? When the team is small (one or two teams), the domain is still being discovered, the product is an early-stage startup, or you lack automated deployment and monitoring. In those cases a well-structured monolith ships faster. Martin Fowler's "Monolith First" advice says: start with a monolith, find the stable boundaries, and extract services only when a real pain (scaling, team contention, release speed) appears.

Q6. What is a modular monolith? A single deployable whose code is split into strongly separated modules, each with its own public API and private internals, and ideally its own database schema. It gives most of the design benefits of microservices (clear boundaries, ownership) without the network and operational costs, and it is the easiest starting point for later extraction. In Spring, **Spring Modulith** verifies module boundaries in tests and offers in-process events between modules. → *More:* Ch 1 — Architectural styles

Q7. ⋆ What is the difference between SOA and microservices? SOA (Service-Oriented Architecture) also splits a system into services, but typically shares databases, uses heavyweight protocols (SOAP, WS-*), and puts routing and business logic into a central Enterprise Service Bus (ESB). Microservices push the opposite way: "smart endpoints, dumb pipes", a database per service, lightweight protocols and decentralised governance. Microservices are often described as "SOA done with finer grains and without the ESB".

Q8. What are the key characteristics of a microservice? Single responsibility around a business capability; owns its data; independently deployable; communicates through a well-defined API; loosely coupled to other services but highly cohesive inside; owned by one team ("you build it, you run it"); designed for failure; and observable (logs, metrics, traces).

Q9. ⋆ What do you mean by cohesion and coupling? Cohesion is how strongly the code inside one module belongs together — high cohesion means a change to one business rule touches only one service. Coupling is how much one module depends on another — low coupling means you can change or deploy one service without changing the others. Good microservices aim for **high cohesion, low coupling**; a design where every feature needs changes to five services is a "distributed monolith".

Q10. ⋆ What is a distributed monolith and how do you avoid it? A system split into many services that still have to be deployed together because they are tightly coupled — shared database tables, chatty synchronous call chains, shared domain libraries, or lock-step API changes. You get all the costs of microservices with none of the benefits. Avoid it with database per service, boundaries drawn around business capabilities, backward-compatible API changes, and asynchronous events where an immediate answer is not needed.

Q11. How big should a microservice be? There is no line count. A useful test: one team can own it, it maps to one bounded context (Q17), it can be rewritten in a couple of weeks, and most changes touch only this service. Too small ("nanoservices") creates chatty calls and distributed transactions; too big recreates the monolith's coordination problems.

Q12. What is Conway's Law and why does it matter here? Conway's Law says a system's structure mirrors the communication structure of the organisation that builds it. So service boundaries and team boundaries should match: one team per service (or a few services). The "inverse Conway manoeuvre" is reorganising teams first to get the architecture you want.

Q13. What does "stateless" mean for a microservice? Stateless vs stateful services? A stateless service keeps no client-specific data in memory between requests — any instance can serve any request, so you can add, remove or restart instances freely behind a load balancer. State (sessions, carts) lives in a database or cache such as Redis. Stateful services (databases, Kafka brokers, game servers) keep data locally and need stable identity, replication and careful scaling; in Kubernetes they run as StatefulSets. → *More:* Ch 1 — Stateless and stateful services

Q14. What are the main components of a microservices architecture? The services themselves plus the platform around them: an API gateway (single entry point), service discovery/registry, a config server, a message broker (Kafka/RabbitMQ), databases per service, a container platform (Docker + Kubernetes), CI/CD pipelines, observability (logs, metrics, traces) and security infrastructure (identity provider, secrets store). Optionally a service mesh for traffic, mTLS and telemetry.

Q15. Name companies that use microservices and why they adopted them. Netflix (moved from a monolith after a 2008 database corruption outage, to scale streaming globally), Amazon (the "two-pizza team" model with service-owned APIs), Uber (split a monolith to scale teams and cities), Spotify (squads owning services). The common reason is scaling the *organisation* — hundreds of engineers shipping independently — not just scaling traffic.

Q16. What is the twelve-factor app methodology? A set of twelve rules for building cloud-ready services: one codebase per app, explicit dependencies, config in the environment, backing services as attached resources, separate build/release/run stages, stateless processes, port binding, scale out via

processes, disposability (fast start, graceful shutdown), dev/prod parity, logs as event streams, and admin tasks as one-off processes. It is the checklist that makes a service easy to containerise and run on Kubernetes. → *More:* Ch 13 — Configuration Externalization

---

2. Decomposition, DDD and migration

Q17. ⋆ What is a bounded context? A bounded context is a term from Domain-Driven Design: a boundary inside which a domain model and its vocabulary have one precise meaning. "Customer" in Billing (payment methods, invoices) is a different model from "Customer" in Shipping (addresses, delivery preferences). Each bounded context is a natural candidate for a microservice, because inside it the language and rules are consistent and it can own its own data.

Q18. ⋆ What is Domain-Driven Design (DDD) and how does it relate to microservices? DDD is an approach to software design that models the code closely on the business domain, built with domain experts using a shared "ubiquitous language". Its strategic part (bounded contexts, context maps, subdomains) tells you where to draw service boundaries; its tactical part (entities, value objects, aggregates, domain events) tells you how to structure the code inside a service. Microservices without DDD-style boundaries tend to end up chatty and tightly coupled.

Q19. ⋆ How would you decompose a monolith into microservices? Decompose by **business capability** (what the business does: ordering, payments, catalogue) or by **subdomain/bounded context**, not by technical layer. Practical steps: run an event-storming workshop to find domain events and contexts; map which data each capability owns; pick an edge capability with few dependencies and high change rate; extract it behind the existing API using the Strangler Fig pattern; split its data last. Repeat, measuring whether deployments really got more independent. → *More:* Ch 2 — Strangler Fig

Q20. ⋆ What is the Strangler Fig pattern? A way to replace a legacy system gradually: put a facade (proxy or gateway) in front of the monolith, build new functionality as separate services, and route one capability at a time from the old system to the new one. Over time the new system "strangles" the old one until it can be switched off. It avoids a risky big-bang rewrite and lets you roll back per route. → *More:* Ch 2 — Strangler Fig

Q21. What is an aggregate in DDD and why does it matter for microservices? An aggregate is a cluster of objects treated as one unit for changes, with one root entity as its only entry point (for example, `Order` with its `OrderLines`). All invariants inside an aggregate are kept in one local transaction; anything across aggregates is eventually consistent. This maps neatly to microservices: a transaction never spans services, and other services refer to an aggregate only by its ID.

Q22. What are entities, value objects and domain events? An entity has identity that persists over time (an `Order` with an ID). A value object is defined only by its values and is immutable (`Money(100, EUR)`, an `Address`) — a Java `record` is a good fit. A domain event records something that happened that the business cares about (`OrderPlaced`), named in the past tense; it is what other services subscribe to.

Q23. What is a context map? Name some relationships between bounded contexts. A context map is a diagram of the bounded contexts and how they relate. Common relationships: **customer–supplier** (downstream depends on upstream, who considers its needs), **conformist** (downstream simply adopts upstream's model), **anti-corruption layer** (downstream translates upstream's model into its own), **shared kernel** (a small shared model both own), **open host service / published language** (upstream publishes a stable public API or schema).

Q24. ⋆ What is an anti-corruption layer? A translation layer between your service and another system (often a legacy monolith or third-party API) so that the other system's model and quirks do not leak into your clean domain model. It converts their data and calls into your concepts and back. It is standard during Strangler Fig migrations. → *More:* Ch 2 — Strangler Fig

Q25. What is event storming? A workshop technique in which developers and domain experts put domain events (orange sticky notes, past tense: "Order Placed", "Payment Captured") on a timeline, then add commands, actors, policies and aggregates. Clusters that emerge suggest bounded contexts and thus service boundaries. It is a fast way to find boundaries before writing code.

Q26. Decompose by business capability vs by subdomain — what is the difference? A business capability is something the business does to create value (manage inventory, take payments) and is fairly stable

over time. A subdomain is DDD's split of the problem space into **core** (your competitive edge — invest most here), **supporting** (needed but specific to you) and **generic** (commodity, often bought, like identity). Both lead to similar boundaries; subdomains add the guidance about where to invest and what to buy.

Q27. What is the "shared database" anti-pattern? Several services reading and writing the same tables. It looks convenient but couples services at the schema level: one team's column change breaks another service, you cannot scale or choose storage per service, and ownership of data rules is unclear. It is acceptable only as a temporary migration step, with a plan to split. → *More:* Ch 10 — Saga, "What is database per service?"

Q28. How do you split the database when migrating from a monolith? Go step by step: first separate the code module and make it the only writer of its tables; then move those tables into a separate schema; then a separate database. Keep data in sync during the transition with change data capture (CDC — streaming the database's change log, e.g. Debezium reading the write-ahead log into Kafka) or with the service's events, and replace cross-table joins with API calls or local read copies. → *More:* Ch 2 — Strangler Fig, "The part the pattern doesn't solve: data"

Q29. What is CDC (Change Data Capture)? CDC streams every insert, update and delete from a database's transaction log to other systems, usually through Kafka, without changing application code. Uses: syncing data during a monolith migration, feeding search indexes and caches, and as the relay in the transactional outbox pattern (Q107). Debezium is the common open-source tool.

Q30. What is the difference between a microservice and a serverless function? A microservice is a long-running process you deploy and scale (usually as containers); a serverless function (AWS Lambda, Cloud Functions) is code the platform starts per event and bills per invocation, scaling to zero. Functions suit spiky, event-driven, short tasks; they bring cold-start latency, execution time limits and vendor lock-in. A microservice can be implemented with several functions, so they are not exclusive. → *More:* Ch 1 — Architectural styles

---

3. Communication: REST, gRPC and messaging

Q31. ⋆ How do microservices communicate with each other? Two styles. **Synchronous** request/response — REST over HTTP or gRPC — where the caller waits for the answer; simple, but couples availability (if the callee is down, the caller fails). **Asynchronous** messaging — events or commands through a broker such as Kafka or RabbitMQ — where the sender does not wait; decoupled and resilient, but eventually consistent and harder to trace. Most systems use sync for queries that need an answer now and async for propagating state changes. → *More:* Ch 1 — Synchronous vs asynchronous communication · Ch 11 — Event-Driven

Q32. ⋆ What is the difference between synchronous and asynchronous communication? Synchronous: the caller sends a request and blocks (or awaits) until the response arrives — the two services must both be up at the same time. Asynchronous: the caller hands a message to a broker and carries on; the receiver processes it later. Async gives temporal decoupling, load levelling (the queue absorbs bursts) and better fault isolation, but adds a broker to operate, eventual consistency, and duplicate/out-of-order message handling. → *More:* Ch 11 — Event-Driven, Part 0

Q33. ⋆ What is REST? What makes an API RESTful? REST (Representational State Transfer) is an architectural style for HTTP APIs: resources are identified by URLs (`/orders/42`), manipulated with standard HTTP methods (GET, POST, PUT, PATCH, DELETE), represented as JSON or similar, and each request is stateless (carries all needed context). Good REST APIs use correct status codes, are cacheable where possible, and use nouns, not verbs, in paths. → *More:* Ch 1 — API styles · Ch 1 — API design basics

Q34. ⋆ REST vs gRPC vs message queues — when do you use each? REST/JSON: public and browser-facing APIs, simple CRUD, easy debugging. gRPC: internal service-to-service calls that need low latency, strong typed contracts and streaming — it uses HTTP/2 and binary Protocol Buffers. Message queues/streams: when the caller does not need an immediate answer, when many consumers must react to one event, or to absorb load spikes. → *More:* Ch 1 — API styles

Q35. What is gRPC and why is it faster than REST/JSON? gRPC is a remote procedure call framework: you define services and messages in a `.proto` file and generate client and server code in many languages.

It is faster because Protocol Buffers are a compact binary format (smaller and cheaper to parse than JSON) and HTTP/2 multiplexes many calls over one connection with header compression. It supports four call types: unary, server streaming, client streaming and bidirectional streaming. Downsides: not natively callable from browsers (needs gRPC-Web or a gateway), harder to inspect by hand.

Q36. What is GraphQL and when would you use it in a microservices system? GraphQL is a query language for APIs where the client asks for exactly the fields it needs in one request, against a typed schema. It suits a client-facing aggregation layer (often a BFF or gateway) over many services, avoiding over-fetching and multiple round trips. Costs: caching is harder (one POST endpoint), expensive queries need depth/complexity limits, and N+1 calls to backends need batching (DataLoader). → *More:* Ch 4 — BFF

Q37. ⋆ Which HTTP methods are idempotent? What is the difference between PUT and PATCH, and PUT and POST? Idempotent means repeating the request has the same effect as doing it once. GET, HEAD, PUT, DELETE (and OPTIONS) are idempotent; POST and PATCH are not guaranteed to be. PUT replaces the whole resource at a known URL; PATCH applies a partial change; POST usually creates a new resource whose ID the server chooses, so retrying a POST can create duplicates unless you add an idempotency key (Q119). → *More:* Ch 1 — HTTP · Ch 8 — Retry, Part 0

Q38. ⋆ How do you version APIs in microservices? Options: URI version (`/v2/orders`), a header (`API-Version: 2`), a query parameter, or a media type (`application/vnd.shop.v2+json`). More important than the mechanism is the rule: make changes backward compatible (add optional fields, never remove or rename without a new version), support old versions for a deprecation period, and announce sunset dates. Spring Framework 7 adds first-class API versioning to Spring MVC and WebFlux (`@GetMapping(version = "2")` with a configured strategy for path, header, query or media type). → *More:* Ch 3 — API Gateway, "Version management"

Q39. What does "backward compatible" mean for an API or event schema? Old clients keep working against the new version. Safe changes: adding optional fields, adding new endpoints, accepting more input. Breaking changes: removing or renaming fields, changing types or meaning, making optional fields required. Consumers should follow the "tolerant reader" rule — ignore fields they do not understand — so producers can add fields freely.

Q40. ⋆ What is a message broker? Kafka vs RabbitMQ? A message broker is middleware that receives messages from producers and delivers them to consumers, decoupling them in time. **RabbitMQ** is a traditional queue broker: smart routing (exchanges, routing keys), messages are removed once acknowledged, good for task queues and complex routing. **Kafka** is a distributed, partitioned, replicated log: messages are retained for a configured time, consumers track their own offset and can replay, ordering is per partition, and throughput is very high — good for event streaming and event sourcing. → *More:* Ch 1 — Message queues · Ch 1 — Event streaming

Q41. Point-to-point queue vs publish-subscribe? In point-to-point, each message is consumed by exactly one consumer among competing workers — good for distributing tasks. In publish-subscribe, each message is delivered to every subscribed group — good for broadcasting events ("OrderPlaced" goes to billing, shipping and analytics). Kafka does both with consumer groups: within one group partitions are split among consumers (point-to-point), and each group gets every message (pub-sub). → *More:* Ch 11 — Event-Driven, "What is publish-subscribe?"

Q42. ⋆ How does Kafka guarantee message ordering? Only within a partition. Messages with the same key go to the same partition (by hash of the key), so key by the entity ID (for example `orderId`) to keep all events for one order in order. There is no ordering across partitions. Retries with more than one in-flight request could reorder messages, which the idempotent producer (on by default in modern Kafka) prevents. → *More:* Ch 11 — Event-Driven · Ch 1 — Event streaming

Q43. What are Kafka partitions, consumer groups and offsets? A topic is split into partitions, each an ordered append-only log; partitions are the unit of parallelism and are replicated across brokers. A consumer group is a set of consumers sharing the work — each partition is read by at most one consumer in the group, so more consumers than partitions sit idle. An offset is a consumer's position in a partition; committing it after processing gives at-least-once delivery. → *More:* Ch 1 — Event streaming

Q44. Events vs commands vs queries in messaging? A command asks one specific receiver to do something (`ReserveStock`) and may be rejected. An event announces that something already happened (`StockReserved`) to anyone interested; the publisher does not know or care who listens. A query asks for data and

changes nothing. Naming them correctly matters: events are past tense and immutable. → *More:* Ch 11 — Event-Driven, "What is an event?" · Ch 12 — CQRS

Q45. Event notification vs event-carried state transfer? An event notification is thin — "Order 42 changed" — and consumers call back the source to get details (simpler, but adds load and coupling). Event-carried state transfer puts the needed data in the event — "Order 42 placed, items…, total…" — so consumers can keep a local copy and never call back (more decoupled and resilient, but bigger events and schema-evolution concerns).

Q46. ⋆ What is a dead-letter queue (DLQ)? A separate queue or topic where messages go after they fail processing a set number of times (for example a malformed "poison" message). It stops one bad message from blocking the whole stream or retrying forever. You alert on DLQ growth, inspect, fix the bug or data, and replay the messages. Spring Kafka's `DefaultErrorHandler` with a `DeadLetterPublishingRecoverer` does this. → *More:* Ch 11 — Event-Driven, "Error handling and message durability"

Q47. What is backpressure? A way for a slow consumer to signal a fast producer to slow down, so the consumer is not overwhelmed and memory does not grow without limit. Pull-based consumers (Kafka) have it naturally; reactive streams (Project Reactor's `Flux`) have it built into the protocol; for HTTP you use rate limits, bounded queues and 429/503 responses.

Q48. How do you manage schema evolution in event-driven systems? Use a schema format with evolution rules (Avro, Protobuf or JSON Schema) and a schema registry that rejects incompatible changes at publish time. Prefer backward-compatible changes (add optional fields with defaults). For a breaking change, publish a new event type or version alongside the old one, migrate consumers, then retire the old one. Consumers should be tolerant readers.

Q49. When would you use WebSockets or Server-Sent Events instead of REST? When the server must push updates to the client in real time — chat, live dashboards, notifications, collaborative editing. WebSockets give a two-way persistent connection; SSE gives a simpler one-way server-to-client stream over HTTP. Costs: long-lived connections are stateful and harder to load-balance and scale, so a dedicated gateway/connection service is common. → *More:* Ch 1 — Real-time communication

Q50. How do you avoid long synchronous call chains? Each hop multiplies latency and failure probability (five services at 99.9% availability each give about 99.5% together). Fixes: let services keep local copies of the data they need (updated by events), aggregate in a gateway/BFF in parallel instead of chaining, move non-urgent steps to async messaging, and set timeouts at every hop with a total deadline. → *More:* Ch 3 — API Gateway

---

4. API Gateway, BFF, service discovery and service mesh

Q51. ⋆ What is an API Gateway and why do we need it? An API gateway is a single entry point that sits between clients and the services. It routes each request to the right service and handles cross-cutting concerns once: authentication, rate limiting, TLS termination, request/response transformation, response aggregation, caching, logging and metrics. Clients then depend on one stable API instead of knowing the internal topology. Examples: Spring Cloud Gateway, Kong, NGINX, AWS API Gateway, Envoy. → *More:* Ch 3 — API Gateway

Q52. ⋆ What are the drawbacks of an API Gateway and how do you mitigate them? It is a potential single point of failure and bottleneck, adds a network hop of latency, and can grow into a "god service" full of business logic owned by nobody. Mitigate by running several stateless gateway instances behind a load balancer across zones, keeping it thin (routing and cross-cutting only, no business rules), using timeouts and circuit breakers on its upstream calls, and giving it clear team ownership. → *More:* Ch 3 — API Gateway, Lesson 4

Q53. ⋆ What is the Backend for Frontend (BFF) pattern? How is it different from an API Gateway? A BFF is a separate backend per client type (web, iOS/Android, TV, partner API), owned by the frontend team, which aggregates and shapes data exactly for that client. A general API gateway is one shared entry point for all clients. They are often combined: a thin gateway handles auth and routing, and BFFs behind it do client-specific aggregation. The cost of BFFs is some duplicated code across them. → *More:* Ch 4 — BFF · Ch 4, "BFF vs. API Gateway"

Q54. What is API composition / the aggregator pattern? A component (gateway, BFF or dedicated service) calls several services, ideally in parallel, and merges the results into one response — for example a product page combining catalogue, price, stock and reviews. It replaces the cross-service join you lost with database per service. Watch out for the slowest dependency dominating latency; use timeouts and return partial results when an optional part fails. → *More:* Ch 3 — API Gateway, "Aggregation of results"

Q55. ⋆ What is service discovery and why is it needed? In containers and auto-scaling environments, service instances start and stop constantly and their IP addresses change. Service discovery lets a caller find a healthy instance by a logical name (`order-service`) instead of a hard-coded address. It needs a **service registry** — a database of live instances that services register with and that is kept fresh by health checks or heartbeats. → *More:* Ch 5 — Service Discovery

Q56. ⋆ Client-side vs server-side service discovery? Client-side: the caller queries the registry, gets the list of instances and load-balances itself (Eureka + Spring Cloud LoadBalancer). Fewer hops, but every client needs the library. Server-side: the caller sends to a router or load balancer that does the lookup (Kubernetes Service with DNS, AWS ALB). Clients stay simple and language-agnostic, at the cost of an extra hop. On Kubernetes, server-side via Services and DNS is the default and you rarely need Eureka. → *More:* Ch 5 — Client-side vs. server-side discovery

Q57. Self-registration vs third-party registration? With self-registration the service registers itself on startup and sends heartbeats (Eureka client). With third-party registration a separate component — the platform — watches instances and registers them (Kubernetes endpoints controller, Consul with a registrator). Third-party keeps service code free of registry logic. → *More:* Ch 5 — Self-registration vs. third-party registration

Q58. ⋆ What is Eureka? Why is Eureka described as AP? Eureka is Netflix's service registry, integrated through Spring Cloud Netflix. In CAP terms (consistency, availability, partition tolerance) it favours availability: during network problems it keeps serving possibly stale instance lists and enters "self-preservation mode" instead of evicting many instances at once. A stale list is survivable because clients retry another instance; a registry that refuses to answer would stop all traffic. → *More:* Ch 5 — Service Discovery · Ch 1 — CAP theorem

Q59. How does service discovery work in Kubernetes? A Kubernetes **Service** gives a stable virtual IP and DNS name (`orders.shop.svc.cluster.local`) in front of a changing set of Pods selected by labels. The control plane tracks ready Pods (via readiness probes) in EndpointSlices, and kube-proxy (or an eBPF data plane) load-balances connections to them. Callers just use the DNS name. → *More:* Ch 5 — Service Discovery · Ch 1 — Kubernetes basics

Q60. ⋆ What is client-side load balancing? Which algorithms exist? The client picks the target instance itself from the list it got from discovery, avoiding a central load balancer hop. In Spring, a `@LoadBalanced` `RestClient.Builder` or `WebClient.Builder` uses Spring Cloud LoadBalancer (which replaced Netflix Ribbon). Common algorithms: round robin, random, least connections, weighted, and zone- or latency-aware choices. → *More:* Ch 1 — Load balancers · Ch 5 — Load balancing strategies

Q61. ⋆ What is a service mesh? When would you use one? A service mesh is an infrastructure layer that handles service-to-service traffic through a proxy next to each service (a sidecar such as Envoy, or a per-node proxy in Istio's "ambient" mode). A control plane (Istio, Linkerd) configures the proxies. It provides mTLS, retries, timeouts, circuit breaking, traffic splitting for canaries, and uniform metrics and traces — without changing application code and in any language. Use it with many services and polyglot teams; it adds latency, resource cost and operational complexity. → *More:* Ch 9 — Sidecar · Ch 1 — Service discovery, sidecars and service mesh

Q62. ⋆ What is the sidecar pattern? A helper process deployed alongside the main application (in Kubernetes, another container in the same Pod) sharing its lifecycle, network and possibly storage. It takes over cross-cutting concerns — proxying, mTLS, log shipping, config reloading — so the application stays focused on business logic and the concern is solved once for every language. → *More:* Ch 9 — Sidecar

Q63. Library (e.g. Resilience4j) vs service mesh for resilience — which is better? A library runs in-process, knows the business context (fallback logic, which calls are idempotent) and costs no extra hop, but must be implemented per language. A mesh is language-agnostic and centrally configured, but sees only network-level information and can retry non-idempotent calls blindly. A common split: mesh

for mTLS, timeouts and traffic shaping; library for fallbacks and business-aware retries. Avoid configuring retries in both places. → *More:* Ch 9 — Sidecar · Ch 8 — Retry, "Placement in the architecture"

Q64. What is the difference between a reverse proxy, a load balancer and an API gateway? A reverse proxy sits in front of servers and forwards client requests to them (TLS termination, caching, hiding servers). A load balancer is a reverse proxy whose main job is spreading load across instances with health checks. An API gateway is a reverse proxy with API-level features: routing by path/version, auth, rate limiting per client, transformation and aggregation. In practice the same product (NGINX, Envoy) can play all three roles. → *More:* Ch 1 — Proxies and API gateways

Q65. Edge (north-south) vs internal (east-west) traffic? North-south traffic enters or leaves the system (clients to the gateway); east-west traffic flows between services inside the cluster. The API gateway governs north-south; a service mesh or internal policies govern east-west. Security for east-west is often forgotten — zero trust says encrypt and authenticate it too (mTLS).

Q66. Should the API gateway do authentication, authorization, or both? Typically the gateway authenticates (validates the token, rejects anonymous calls) and does coarse checks (scopes, rate limits per client). Fine-grained authorization ("may this user cancel *this* order?") belongs in the owning service, which knows the data. Each service should still validate the token it receives rather than blindly trusting the network. → *More:* Ch 3 — API Gateway, "Security and authentication"

---

5. Resilience and fault tolerance

Q67. ⋆ How do you handle failures in microservices / what is fault tolerance? Assume every remote call can fail or be slow. Combine: timeouts on every call, retries with backoff for transient errors, circuit breakers to stop calling a failing dependency, bulkheads to isolate resources, fallbacks or graceful degradation, rate limiting and load shedding to protect yourself, async messaging to decouple, and redundancy (several instances across zones). Then verify with chaos testing. → *More:* Ch 6, Ch 7, Ch 8 · Ch 1 — Faults, failures and fault tolerance

Q68. ⋆ What is a cascading failure? A failure that spreads: service C becomes slow, so B's threads waiting on C pile up until B runs out of threads and stops responding, then A, which calls B, does the same — and the whole system goes down because of one slow dependency. Timeouts, circuit breakers and bulkheads exist mainly to break this chain. → *More:* Ch 6 — Circuit Breaker, "Cascading failures, step by step"

Q69. ⋆ Explain the circuit breaker pattern and its states. A circuit breaker wraps calls to a dependency and tracks failures. **Closed**: calls flow normally and failures are counted in a sliding window. When the failure (or slow-call) rate passes a threshold it goes **Open**: calls fail immediately (or go to a fallback) without touching the dependency, giving it time to recover. After a wait it goes **Half-Open**: a few trial calls are allowed; if they succeed it closes, otherwise it opens again. → *More:* Ch 6 — Circuit Breaker, "The three states"

Q70. ⋆ Why is a slow service often worse than a dead one? A dead service fails fast (connection refused) so the caller frees its thread at once. A slow service holds the caller's threads and connections until the timeout, so they pile up and exhaust the caller's pools — that is how cascades start. That is why circuit breakers should also count *slow calls* (Resilience4j `slowCallRateThreshold`) and every call needs a timeout. → *More:* Ch 6 — Circuit Breaker

Q71. ⋆ What is the retry pattern? What should and should not be retried? Automatically repeat a failed call in the hope that the failure was transient. Retry timeouts, connection resets, 503/502/504, 429 (honouring `Retry-After`). Do not retry 400, 401, 403, 404, 409 or validation errors — they will fail again. Only retry operations that are idempotent, or make them idempotent with an idempotency key. → *More:* Ch 8 — Retry, "Transient vs. permanent"

Q72. ⋆ What is exponential backoff with jitter and why use it? Exponential backoff waits longer after each failure (for example 100 ms, 200 ms, 400 ms, up to a cap) to give the dependency room to recover. Jitter adds randomness to each wait ("full jitter": random between 0 and the backoff), so that thousands of clients that failed together do not all retry at the same instant — the thundering-herd

problem. → *More:* Ch 8 — Retry, "Backoff strategy" and "Jitter" · Ch 1 — Timeouts, retries, exponential backoff and jitter

Q73. ⋆ What is a retry storm and how do you avoid it? Retries multiplying load on an already struggling service. If three layers each retry three times, one user request can become 27 calls to the bottom service. Avoid it by retrying at one layer only (usually the one closest to the failing call or at the edge), capping attempts, using backoff with jitter, a retry budget (retries limited to, say, 10% of traffic), and a circuit breaker that stops retries when the dependency is clearly down. → *More:* Ch 8 — Retry, "Retry storms and thundering herd"

Q74. ⋆ What is the bulkhead pattern? Partition resources (thread pools, connection pools, semaphores, even whole service instances) per dependency or per type of work, like watertight compartments in a ship's hull. If the recommendation service hangs, it can exhaust only its own pool of, say, 10 threads, while checkout keeps its own pool and continues to work. → *More:* Ch 7 — Bulkhead

Q75. ⋆ Circuit breaker vs bulkhead vs retry — how do they work together? Retry bounds *how many* attempts; circuit breaker bounds *how long* you keep calling a failing dependency; bulkhead bounds *how much* of your resources one dependency can consume. A typical order (outermost first, Resilience4j default aspect order): Retry → CircuitBreaker → RateLimiter → TimeLimiter → Bulkhead → the call. Each retry attempt then goes through the breaker, so an open breaker stops retries quickly. → *More:* Ch 7 — Bulkhead · Ch 8 — Retry, "Circuit breaker integration"

Q76. ⋆ Why is a timeout essential on every remote call? How do you choose its value? Without a timeout a hung dependency holds your thread forever. Choose from the dependency's measured latency: slightly above its p99 (e.g. p99 = 300 ms → timeout 500 ms), never the library's default (often infinite or very long). Set both connect and read timeouts, and propagate a total deadline so downstream calls do not keep working after the upstream caller has already given up. → *More:* Ch 1 — Timeouts, retries, exponential backoff and jitter · Ch 6 — "What is a timeout?"

Q77. What is deadline propagation? The original caller sets a total time budget (say 2 s) and each service passes the *remaining* budget downstream (gRPC does this with deadlines; with HTTP you use a header). A service that receives a request with too little time left fails fast instead of doing work nobody will wait for.

Q78. ⋆ What is a fallback? Give examples. What you return when the real call fails or the breaker is open: cached or last-known-good data, a default value (generic recommendations instead of personalised ones), an empty but valid response, or queuing the action for later ("we will email your receipt"). Fallbacks must be cheap and must not themselves call the same failing dependency. For critical data (payments, balances) the honest fallback is often a clear error. → *More:* Ch 6 — "What is a fallback?" · Ch 1 — Circuit breakers, bulkheads and fallbacks

Q79. ⋆ What is rate limiting and which algorithms are used? Rate limiting caps how many requests a client may make in a time window, protecting services from overload and abuse; excess requests get HTTP 429. Algorithms: **token bucket** (tokens refill at a fixed rate, each request takes one, allows short bursts — the most common), **leaky bucket** (smooth constant output), **fixed window counter** (simple, but bursts at window edges), **sliding window log/counter** (accurate, a bit more costly). → *More:* Ch 1 — Rate limiting and throttling · Ch 3 — API Gateway, "Rate limiting challenges"

Q80. How do you rate-limit across many gateway instances? In-memory counters only limit per instance, so N instances allow N times the limit. Use a shared store — typically Redis with an atomic Lua script implementing the token bucket (Spring Cloud Gateway's `RequestRateLimiter` with `RedisRateLimiter` does this). Alternatively accept approximate limits by dividing the quota by the instance count. → *More:* Ch 3 — API Gateway

Q81. Rate limiting vs throttling vs load shedding? Rate limiting rejects a *client* above its quota (fairness, abuse). Throttling slows down or queues requests rather than rejecting them. Load shedding is the server protecting *itself*: when overall load is above capacity it drops low-priority work (e.g. analytics calls) to keep critical paths (checkout) alive. → *More:* Ch 1 — Load shedding and graceful degradation

Q82. What is graceful degradation? Keeping the core function working with reduced features when parts fail: the shop still sells when the recommendation or review service is down, showing a page without those sections. It requires deciding upfront which dependencies are critical and which are optional, and coding fallbacks for the optional ones. → *More:* Ch 1 — Load shedding and graceful degradation

Q83. ⋆ Show how you would add a circuit breaker and retry to a Spring Boot call. With Resilience4j's Spring Boot integration:

```
@CircuitBreaker(name = "inventory", fallbackMethod = "stockFallback")
@Retry(name = "inventory")
public Stock stock(String sku) {
    return inventoryClient.getStock(sku);   // RestClient / HTTP interface call
}
private Stock stockFallback(String sku, Throwable t) {
    return Stock.unknown(sku);              // degrade, do not fail the page
}
```

Thresholds go in `application.yml` under `resilience4j.circuitbreaker.instances.inventory` (e.g. `failureRateThreshold: 50`, `slidingWindowSize: 20`, `waitDurationInOpenState: 10s`). The fallback must have the same return type plus a `Throwable` parameter. → *More:* Ch 6 — Circuit Breaker · Ch 8 — Retry

Q84. What resilience features are built into Spring Framework 7 itself? Spring Framework 7 adds `@Retryable` (with backoff, jitter and include/exclude exception types) and `@ConcurrencyLimit` (a simple bulkhead) in core, enabled with `@EnableResilientMethods`, plus a programmatic `RetryTemplate` in `org.springframework.core.retry`. By default `@Retryable` retries any exception 3 times with a 1 s delay. They replace the separate Spring Retry project (Spring Boot 4 no longer manages its version) and cover retry and concurrency throttling without an extra dependency; for circuit breakers, rate limiters and time limiters you still use Resilience4j (often via Spring Cloud Circuit Breaker).

Q85. ⋆ What is idempotency and why is it critical in microservices? An operation is idempotent if doing it twice has the same effect as doing it once. Because retries, at-least-once message delivery and user double-clicks all cause duplicates, every handler that changes state must tolerate duplicates — otherwise a retry can charge a customer twice. Techniques: idempotency keys, natural keys with unique constraints, a processed-message table, and conditional updates (`UPDATE … WHERE version = ?`). → *More:* Ch 8 — Retry, "Idempotency and side effects" · Ch 1 — Delivery guarantees, idempotency and retries

Q86. How do you tune circuit breaker parameters? Base them on data: the dependency's normal error rate and latency percentiles. Set the minimum number of calls high enough that one or two failures at low traffic do not trip it; set the failure-rate threshold well above the normal error rate (e.g. 50%); set the slow-call threshold just above normal p99; set the open-state wait to roughly how long the dependency takes to recover (5–30 s). Alert on state transitions to Open — a breaker that hides errors silently is dangerous. → *More:* Ch 6 — Circuit Breaker, "Performance implications"

Q87. What is chaos engineering? Deliberately injecting failures (killing instances, adding latency, cutting networks, failing a zone) in a controlled way, ideally in production with a limited blast radius, to verify that the system's resilience mechanisms actually work. Netflix's Chaos Monkey popularised it; tools include Chaos Mesh, LitmusChaos, Gremlin and AWS Fault Injection Service. Start with a hypothesis ("if one payment instance dies, p99 stays below 500 ms") and measure.

Q88. How do you size a bulkhead or a thread pool for a dependency? Use Little's law: concurrent requests in flight = arrival rate × latency. If you call a dependency 200 times per second at 50 ms each, about 10 calls are in flight; add headroom (e.g. 15–20). Cap it at what the dependency can handle, and set a small queue or none, so excess fails fast rather than waiting. → *More:* Ch 7 — Bulkhead · Ch 1 — Little's law

Q89. Do virtual threads remove the need for bulkheads and timeouts? No. Virtual threads (final since Java 21; enable in Spring Boot with `spring.threads.virtual.enabled=true`) make blocking calls cheap, so you rarely run out of *threads*. But the downstream service, connection pools and database still have limits — without a concurrency cap you can now flood them with thousands of simultaneous calls. Keep timeouts, and use semaphore-style bulkheads (e.g. `@ConcurrencyLimit`) instead of thread-pool bulkheads.

---

6. Data management: database per service, saga, outbox, CQRS, event sourcing

Q90. ⋆ What is the database-per-service pattern? Why should each microservice have its own database? Each service owns its data store privately; other services can reach that data only through the

service's API or events, never by querying its tables. This keeps services loosely coupled (schema changes stay internal), lets each pick the right storage (PostgreSQL, MongoDB, Redis, Elasticsearch — "polyglot persistence"), and allows independent scaling. "Own database" can mean a separate schema on a shared server; what matters is that no other service touches it. → *More:* Ch 10 — Saga, "Database per service"

Q91. ⋆ What problems does database per service create? No cross-service ACID transactions, no SQL joins across services, data duplication, and eventual consistency between copies. Answers: sagas for multi-service business transactions, API composition or CQRS read models for queries across services, events plus the outbox pattern to keep copies in sync. → *More:* Ch 10 — Saga · Ch 12 — CQRS

Q92. ⋆ How do you manage transactions across multiple microservices? Avoid them where possible by drawing boundaries so one business invariant lives in one service. When a business process must span services, use a **saga**: a sequence of local transactions, each publishing an event or message that triggers the next, with compensating transactions to undo completed steps if a later step fails. Use the transactional outbox so each step's state change and its message are atomic. Two-phase commit is generally avoided. → *More:* Ch 10 — Saga · Ch 1 — Distributed transactions

Q93. ⋆ What is the Saga pattern? A saga implements a business transaction that spans services as a series of local transactions. Each step commits in its own service's database. If step *n* fails, the saga runs compensating transactions for steps *n-1* … 1 in reverse (refund the payment, release the stock). It gives eventual consistency, not isolation: other requests can see the intermediate states. → *More:* Ch 10 — Saga

Q94. ⋆ Choreography vs orchestration sagas? In **choreography** each service listens for events and decides what to do next ("OrderCreated" → payment charges → "PaymentCompleted" → inventory reserves); there is no central coordinator, which is simple for 2–4 steps but the flow is hard to see and cyclic dependencies creep in. In **orchestration** a central orchestrator (a service or a workflow engine such as Temporal or Camunda) tells each participant what to do and tracks state; the flow is explicit, testable and easier to monitor, at the cost of a central component. → *More:* Ch 10 — Saga, "The two types"

Q95. ⋆ What is a compensating transaction? Why is it not the same as a rollback? A compensating transaction is a new business action that semantically undoes an earlier committed step: "refund payment" for "charge payment", "release seats" for "reserve seats". It is not a rollback because the original step already committed and others may have seen it; some effects cannot be undone (an email was sent), so compensation may be "send a correction email". Compensations must be idempotent and must eventually succeed (retried until they do). → *More:* Ch 10 — Saga, "Compensating transactions"

Q96. ⋆ What is two-phase commit (2PC) and why is it avoided in microservices? 2PC is a protocol where a coordinator asks all participants to *prepare* (lock resources and vote) and then tells all to *commit* or *abort*. It gives atomicity across databases but holds locks during the protocol, blocks if the coordinator fails at the wrong moment, needs every participant to support XA transactions (Kafka and most NoSQL stores do not), and couples availability — one slow participant stalls everyone. Sagas trade isolation for availability and loose coupling. → *More:* Ch 1 — Distributed transactions · Ch 10 — Saga

Q97. Sagas have no isolation. What anomalies can happen and how do you deal with them? Because each step commits independently, other transactions can see or change intermediate data: lost updates, dirty reads (reading an order that will be cancelled), fuzzy reads. Countermeasures: **semantic locks** (a status like `PENDING` that other operations respect), **commutative updates** (operations that can apply in any order), re-reading values before acting, ordering steps so that risky or irreversible ones come last (pivot transaction), and versioning.

Q98. What are compensatable, pivot and retriable transactions in a saga? Compensatable steps come first and can be undone. The pivot transaction is the go/no-go point (e.g. charging the card): if it succeeds, the saga will complete. Retriable steps come after the pivot and must not fail permanently — they are retried until they succeed (e.g. sending the shipping request). Ordering steps this way minimises the compensation you need.

Q99. ⋆ What is the transactional outbox pattern? It solves the **dual-write problem**: a service must update its database *and* publish an event, but you cannot make a database commit and a Kafka send atomic. With the outbox, the service writes the business change and an `outbox` row containing the event in the same local database transaction. A separate relay — a poller, or CDC like Debezium reading the

database log — publishes outbox rows to the broker and marks them sent. Delivery becomes at-least-once, so consumers must be idempotent. → *More:* Ch 10 — Saga · Ch 11 — Event-Driven

```
BEGIN;
UPDATE orders SET status = 'PAID' WHERE id = 42;
INSERT INTO outbox (id, aggregate_id, type, payload)
VALUES (gen_random_uuid(), 42, 'OrderPaid', '{"orderId":42}');
COMMIT;   -- both or neither; the relay publishes the outbox row later
```

Q100. ⋆ What is the dual-write problem? Writing to two systems (database and message broker, or two databases) without a shared transaction. If the process crashes between the two writes, they disagree: the order is saved but no event is published, or the event is published but the save rolled back. Publishing "after commit" in code does not fix it — a crash still loses the event. Solutions: transactional outbox, CDC, or event sourcing (the event *is* the write).

Q101. What is the inbox pattern / idempotent consumer? The consumer stores the IDs of messages it has processed in a table, in the same transaction as the business change. When a duplicate arrives (at-least-once delivery), it finds the ID already there and skips it. A unique constraint on `message_id` makes this race-safe. Together, outbox (producer side) and inbox (consumer side) give "effectively once" processing. → *More:* Ch 1 — Delivery guarantees, idempotency and retries

Q102. ⋆ What is CQRS? Command Query Responsibility Segregation: use separate models for writes (commands that change state and enforce business rules) and reads (queries that return data shaped for screens). It can be just separate classes over one database, or separate databases where the write side publishes events that update one or more denormalised read models (e.g. in Elasticsearch or Redis). It shines when reads vastly outnumber writes or need very different shapes; the cost is complexity and, with separate stores, eventually consistent reads. → *More:* Ch 12 — CQRS

Q103. ⋆ What is event sourcing? How is it different from traditional CRUD? In CRUD you store the current state and overwrite it on each update. In event sourcing you store the full sequence of events that happened to an entity (`AccountOpened`, `MoneyDeposited`, `MoneyWithdrawn`) in an append-only event store, and rebuild current state by replaying them. Benefits: a complete audit log, time travel and debugging, easy creation of new read models, and no dual-write problem. Costs: querying needs projections (usually CQRS), event schemas live forever, and it is unfamiliar to most teams. → *More:* Ch 12 — CQRS, "Level 3 — Event Sourcing" · Ch 11 — Event-Driven, "Event Sourcing and CQRS"

Q104. ⋆ Does CQRS require event sourcing (or vice versa)? No. CQRS can run on one database with separate read and write models, or two databases synced by CDC or events. Event sourcing almost always needs CQRS, because an event log cannot be queried efficiently and you build projections for reads. Treat them as separate decisions and adopt each only when its problem is real. → *More:* Ch 12 — CQRS, "Architecture: three levels"

Q105. What are snapshots in event sourcing? Periodically saving the computed state of an aggregate (say every 100 events) so that loading it means reading the latest snapshot plus only the events after it, instead of replaying thousands of events. They are an optimisation; the event log stays the source of truth.

Q106. How do you query data that lives in several services? Three options: **API composition** — call each service and join in memory (simple, but latency and partial-failure issues); **CQRS read model / materialized view** — a service subscribes to the others' events and keeps a denormalised local copy built for the query (fast, eventually consistent); **data replication** of just the fields you need into your own database. For reporting, stream everything into a data warehouse instead of querying services. → *More:* Ch 12 — CQRS · Ch 3 — API Gateway

Q107. How do you implement the outbox relay — polling vs CDC? Polling: a scheduled job selects unsent rows (`SELECT … FOR UPDATE SKIP LOCKED` so several instances do not grab the same rows), publishes them in order, and marks them sent; simple but adds database load and latency. CDC: Debezium tails the write-ahead log and publishes outbox inserts with low latency and no polling, but adds Kafka Connect to operate. Either way, clean up old rows and keep per-aggregate ordering.

Q108. Can you use Kafka transactions instead of an outbox? Kafka transactions make a consume-transform-produce loop *within Kafka* atomic (exactly-once processing between topics). They do not include your

PostgreSQL commit, so they do not solve the database-plus-event dual write. For that you still need the outbox, CDC or event sourcing.

Q109. How do you handle shared reference data (e.g. country codes, product catalogue) across services? Make one service the owner and publish changes as events; other services keep a read-only local copy updated from those events or cache it with a TTL. For truly static data (currency codes), a versioned shared library or config is acceptable. Never let several services write the same reference tables.

Q110. What is polyglot persistence? Using different storage technologies for different services according to their access patterns: a relational database for orders (transactions), a document store for the catalogue (flexible schema), a search engine for product search, a key-value store for sessions, a graph database for recommendations. Database per service makes it possible; the cost is more technologies to operate and know. → *More:* Ch 1 — Choosing a database

Q111. How do you generate unique IDs across services? Use IDs that need no central coordination: UUIDs (prefer time-ordered UUIDv7 for better index locality), Snowflake-style 64-bit IDs (timestamp + machine ID + sequence), or ULIDs. Avoid a shared database sequence across services — it couples them and becomes a bottleneck. → *More:* Ch 1 — Unique ID generation

Q112. How should a microservice cache data from another service? Cache-aside with a TTL matched to how stale the data may be, invalidated early by the owner's change events if freshness matters. Cache only data you can tolerate being stale (product names, yes; account balance, usually no), and protect against stampedes when hot keys expire (request coalescing, jittered TTLs). → *More:* Ch 1 — Caching strategies · Ch 1 — Cache invalidation

---

7. Consistency, delivery guarantees and ordering

Q113. ⋆ What is eventual consistency? A guarantee that, if no new updates are made, all copies of the data will become consistent after some (usually short) time. Between update and convergence, different services or replicas may return different values. It is the price of independent services and asynchronous messaging; the design question is how long the window is and whether the business can tolerate it (an order showing "processing" for two seconds is fine; overdrawing an account is not). → *More:* Ch 10 — Saga, "Eventual consistency" · Ch 1 — Consistency models

Q114. ⋆ How do you ensure data consistency across microservices? Keep each invariant inside one service's transaction where possible; use sagas with compensations for multi-service processes; publish events reliably with the outbox; make consumers idempotent; use optimistic locking (version columns) for concurrent updates; and add reconciliation jobs that compare systems and fix drift. Accept and design for eventual consistency in the UI (pending states, "we will notify you"). → *More:* Ch 10 — Saga · Ch 11 — Event-Driven, "Data consistency"

Q115. ⋆ Explain the CAP theorem. How does it apply to microservices? CAP says that during a network partition a distributed data store must choose between consistency (every read sees the latest write) and availability (every request gets a non-error answer). Since partitions happen, the real choice is CP or AP behaviour during them. PACELC extends it: else (no partition) you choose between latency and consistency. In microservices, most cross-service data is AP and eventually consistent; things like payments or inventory reservations may need CP within their owning service. → *More:* Ch 1 — CAP theorem and PACELC

Q116. ACID vs BASE? ACID (Atomicity, Consistency, Isolation, Durability) describes strong guarantees of a single database transaction. BASE (Basically Available, Soft state, Eventually consistent) describes systems that stay available and let state converge over time. Inside a service you usually get ACID from its database; across services you design for BASE. → *More:* Ch 1 — Transactions and ACID · Ch 1 — SQL vs NoSQL and BASE

Q117. ⋆ At-most-once, at-least-once and exactly-once delivery — what is the difference? At-most-once: a message may be lost but never duplicated (commit offset before processing). At-least-once: never lost but may be duplicated (commit after processing) — the common default. Exactly-once delivery is impossible in general across a network; what systems provide is exactly-once *processing* — at-least-once delivery plus idempotent handling or transactional deduplication (Kafka transactions within Kafka, inbox tables with a database). → *More:* Ch 1 — Delivery guarantees, idempotency and retries

Q118. How do you handle out-of-order events? Keep ordering where it matters by partitioning on the entity key (one partition per entity's stream). Include a version or sequence number in each event and have consumers ignore events older than what they have already applied. Design handlers to be commutative where possible, and use timestamps only with care because clocks on different machines drift. → *More:* Ch 1 — Clocks and ordering

Q119. ⋆ How do you make a payment API safe to retry (idempotency key)? The client generates a unique idempotency key (a UUID) once per logical operation and sends it in a header on every retry. The server stores the key with the result in the same transaction as the charge, guarded by a unique constraint. A repeated request with the same key returns the stored result instead of charging again; a concurrent duplicate fails the constraint and waits or gets a 409. Keys expire after a retention period (e.g. 24 h). → *More:* Ch 8 — Retry, "Idempotency revisited"

Q120. What is optimistic locking and why is it useful between services? Each record carries a version number; an update succeeds only if the version is unchanged (`UPDATE … SET …, version = version + 1 WHERE id = ? AND version = ?`). If someone else updated it first, zero rows change and the caller retries with fresh data. In JPA it is the `@Version` field. It avoids holding database locks across slow network calls, and ETags with `If-Match` bring the same idea to REST APIs.

Q121. What is read-your-writes consistency and how do you provide it with CQRS? The guarantee that a user sees their own update immediately even if the system is eventually consistent. With an async read model the user might save and then see old data. Options: return the new state in the command's response and update the UI optimistically; read from the write model for that user for a short time; or have the query wait until the read model has reached the version returned by the command. → *More:* Ch 12 — CQRS, "Managing data consistency"

Q122. What is a reconciliation job and why do you still need one? A periodic process that compares data across systems (orders vs payments vs the payment provider's report) and fixes or flags mismatches. Even with sagas and outboxes, bugs, manual fixes and third-party failures cause drift; reconciliation is the safety net that makes eventual consistency trustworthy, especially for money.

Q123. What is the "two generals" / why can't you be sure a remote call failed? When a call times out you do not know whether the request never arrived, was processed and the reply was lost, or is still running. That is why retries need idempotency and why a timeout must be treated as "unknown", not "failed". For important actions, query the status or rely on reconciliation before compensating.

---

8. Observability: logs, metrics and tracing

Q124. ⋆ How do you monitor microservices? What are the three pillars of observability? Observability is the ability to understand a system's internal state from what it emits. The three pillars: **logs** (timestamped records of discrete events, structured as JSON), **metrics** (numeric time series such as request rate, error rate, latency percentiles, CPU), and **traces** (the path and timing of one request across all services). Typical stack: Micrometer in the app, Prometheus + Grafana for metrics, Loki or ELK for logs, OpenTelemetry + Tempo/Jaeger for traces, plus alerting. → *More:* Ch 1 — Observability: logs, metrics and traces

Q125. ⋆ What is distributed tracing and how does it work? It follows one request through every service it touches. The first service creates a **trace ID**; each unit of work (an HTTP call, a DB query) is a **span** with its own span ID, parent span ID, timing and tags. The trace context is propagated in headers (the W3C `traceparent` header is the standard) and through message headers for async calls. Spans are exported to a backend (Jaeger, Zipkin, Tempo) that draws the request as a waterfall, so you see which hop was slow or failed. → *More:* Ch 1 — Observability · Ch 9 — Sidecar

Q126. ⋆ What is a correlation ID? A unique ID attached to a request at the edge and passed through every service call and message, and written into every log line. It lets you search all logs for one user request across services. Today the trace ID from distributed tracing usually plays this role; Spring Boot with Micrometer Tracing puts `traceId` and `spanId` into the logging MDC (Mapped Diagnostic Context) automatically.

Q127. What is OpenTelemetry? OpenTelemetry (OTel) is the vendor-neutral CNCF standard for producing and exporting traces, metrics and logs: APIs, SDKs, auto-instrumentation agents, the OTLP wire protocol and

the Collector (which receives, processes and forwards telemetry to any backend). Instrument once with OTel and switch backends freely. In Spring Boot, Micrometer Tracing bridges to OpenTelemetry, and Spring Boot 4 also ships a `spring-boot-starter-opentelemetry`. → *More:* Ch 1 — Observability

Q128. ⋆ What is centralised logging and why is it needed? With dozens of services and short-lived containers, you cannot SSH into machines to read log files. Each service writes structured logs to stdout; an agent (Fluent Bit, the OTel Collector) ships them to a central store (Elasticsearch/OpenSearch, Loki, a cloud logging service) where you can search by trace ID, service and time. Spring Boot supports structured JSON logging out of the box (`logging.structured.format.console=ecs` or `logstash`).

Q129. What metrics would you track for a microservice? What are the RED and USE methods? RED for request-driven services: Rate (requests per second), Errors (failed requests per second), Duration (latency distribution, especially p95/p99). **USE** for resources: Utilisation, Saturation (queue lengths, thread pool waits), Errors. Add business metrics (orders per minute), JVM metrics (heap, GC pauses), connection pool usage, consumer lag for Kafka, and circuit breaker state. Google's "four golden signals" are latency, traffic, errors and saturation. → *More:* Ch 1 — Percentiles, tail latency and why averages lie

Q130. ⋆ What is Spring Boot Actuator? Actuator adds production-ready endpoints to a Spring Boot app: `/actuator/health` (with liveness and readiness groups for Kubernetes), `/actuator/metrics` and `/actuator/prometheus` (via Micrometer), `/actuator/info`, `/actuator/env`, `/actuator/loggers` (change log levels at runtime) and more. Expose only what you need over HTTP and secure the rest.

Q131. What are SLI, SLO and SLA? What is an error budget? An SLI (service level indicator) is a measured value, e.g. the percentage of requests faster than 300 ms. An SLO (objective) is the target for it, e.g. 99.9% over 30 days. An SLA (agreement) is a contract with customers with penalties, set looser than the SLO. The error budget is 100% minus the SLO (0.1% ≈ 43 minutes per month): while budget remains you ship features; when it is spent you prioritise reliability. → *More:* Ch 1 — SLA, SLO, SLI and error budgets

Q132. How do you alert well in a microservices system? Alert on user-visible symptoms (SLO burn rate, error rate, latency at the edge) rather than every internal cause (high CPU on one Pod), so alerts are few and actionable. Each alert should link to a runbook and dashboard. Use multi-window burn-rate alerts to catch both fast and slow budget burns, and route alerts to the owning team. → *More:* Ch 1 — Dashboards, alerting, on-call and SLO-based alerts

Q133. What is semantic monitoring / synthetic monitoring? Running scripted fake user journeys (log in, search, add to cart) against production continuously and alerting when they fail or get slow. It checks the system works as a whole from the user's point of view, catching problems that per-service metrics miss.

Q134. What is the difference between liveness and readiness health checks? Liveness answers "is the process stuck?" — if it fails, Kubernetes restarts the container. Readiness answers "can it take traffic now?" — if it fails, the Pod is removed from the Service's endpoints but not restarted (e.g. during warm-up or when a critical dependency is unavailable). Do not put downstream dependencies in liveness checks, or one database blip restarts every Pod. A **startup probe** protects slow-starting apps from liveness kills. → *More:* Ch 5 — Health checks

Q135. How do you trace requests across asynchronous messaging? Put the trace context into message headers when producing and extract it when consuming, creating a consumer span linked to the producer span. Spring Kafka and Spring AMQP do this automatically with observation enabled (`spring.kafka.template.observation-enabled=true` and the listener equivalent). With an outbox, store the trace context in the outbox row so the relay can propagate it. → *More:* Ch 11 — Event-Driven

Q136. What is trace sampling and why is it needed? Recording every span of every request is expensive at high traffic, so you keep only a fraction. **Head sampling** decides at the start (e.g. 10% of traces) — cheap but may miss rare errors. **Tail sampling** (in the OTel Collector) decides after the trace finishes, keeping all errors and slow traces and a small sample of normal ones. Spring Boot's `management.tracing.sampling.probability` sets head sampling (default 0.1).

---

9. Deployment: containers, Kubernetes, CI/CD and release strategies

Q137. ⋆ Why are Docker containers used for microservices? A container packages a service with its runtime and dependencies into an image that runs the same on a laptop, in CI and in production. Containers start in seconds, use fewer resources than virtual machines (they share the host kernel), isolate services from each other, and give a uniform unit for orchestration. Spring Boot can build images without a Dockerfile (`./mvnw spring-boot:build-image` using Cloud Native Buildpacks). → *More:* Ch 1 — Virtual machines, containers and Docker

Q138. ⋆ Why is Kubernetes used for microservices? Name its core objects. Kubernetes is a container orchestrator: it schedules containers on a cluster, restarts failed ones, scales them, does rolling updates and provides service discovery and config. Core objects: **Pod** (one or more containers sharing network), **Deployment** (desired replicas, rolling updates), **Service** (stable name and load balancing), **Ingress/Gateway API** (HTTP routing from outside), **ConfigMap** and **Secret** (configuration), **HorizontalPodAutoscaler**, **StatefulSet** (stateful workloads), **Namespace** (isolation). → *More:* Ch 1 — Kubernetes basics

Q139. ⋆ What deployment strategies do you know? Blue-green vs canary vs rolling? Rolling: replace instances a few at a time — no extra capacity, but old and new versions run together. **Blue-green**: run the new version (green) next to the old (blue) and switch all traffic at once; instant rollback by switching back, but doubles capacity during the switch. **Canary**: send a small share of traffic (1%, 5%, 25%…) to the new version, watch metrics, then expand or roll back; safest for catching real-traffic bugs. Also **recreate** (stop all, start new — downtime) and **shadow/dark launch** (copy traffic to the new version without returning its responses). → *More:* Ch 1 — Deployment strategies and rollback

Q140. How do you do a canary release in Kubernetes? Basic: two Deployments behind one Service with replica counts in the desired ratio (coarse). Better: weighted routing in a service mesh (Istio VirtualService), the Gateway API (`HTTPRoute` weights) or an ingress controller, automated with Argo Rollouts or Flagger, which shift traffic in steps and roll back automatically when metrics such as error rate or p99 degrade. → *More:* Ch 1 — Deployment strategies and rollback

Q141. ⋆ How do you achieve zero-downtime deployment? Run several replicas and use rolling or blue-green updates; set readiness probes so new Pods get traffic only when ready; implement graceful shutdown (Spring Boot's `server.shutdown=graceful` finishes in-flight requests after SIGTERM, with a `preStop` delay so the load balancer stops sending first); keep APIs and database schemas backward compatible across the two versions running side by side; use a PodDisruptionBudget.

Q142. ⋆ How do you handle database schema changes with zero downtime? Use the **expand-and-contract** (parallel change) pattern: (1) expand — add the new column or table in a backward-compatible way; (2) deploy code that writes both old and new and reads new with fallback; (3) backfill data; (4) switch reads fully to new; (5) contract — remove the old column in a later release. Run migrations with Flyway or Liquibase as part of the deployment. Never rename or drop in the same release as the code change.

Q143. What is the difference between deployment and release? What are feature flags? Deployment puts new code into production; release exposes the feature to users. Feature flags (toggles checked at runtime) separate the two: you deploy dark, then turn the feature on for internal users, a percentage, or a region, and turn it off instantly if it misbehaves. Flags need cleanup afterwards, or they become technical debt. → *More:* Ch 2 — Strangler Fig, "What is a feature flag?"

Q144. ⋆ What does CI/CD look like for microservices? Each service has its own pipeline triggered by its own repository or path: build, unit tests, contract tests, static analysis and security scanning, build and sign a container image, deploy to staging, run integration/smoke tests, then progressive delivery to production (canary) with automated rollback. GitOps (Argo CD, Flux) keeps the desired cluster state in Git and syncs it. Independence of pipelines is the point — no shared release train. → *More:* Ch 1 — CI/CD

Q145. Monorepo vs polyrepo for microservices? A monorepo puts all services in one repository: easier atomic cross-service changes, shared tooling and code search, but needs build tooling that only builds what changed (Bazel, Nx, Gradle) and discipline to avoid hidden coupling. Polyrepo gives each service its own repository: clear ownership and independent pipelines, but cross-cutting changes and dependency upgrades need many PRs. Both work; the boundary discipline matters more.

Q146. How does autoscaling work for microservices on Kubernetes? The HorizontalPodAutoscaler adds or removes Pods based on metrics (CPU, memory, or custom ones like requests per second or Kafka consumer lag); KEDA scales on event-source metrics including down to zero. The Cluster Autoscaler or Karpenter adds nodes when Pods cannot be scheduled. Scaling only works if services are stateless and start quickly; set resource requests correctly, or HPA's percentages are meaningless. → *More:* Ch 1 — Autoscaling

Q147. What is Infrastructure as Code (IaC)? Defining infrastructure (networks, clusters, databases, DNS) in version-controlled files — Terraform/OpenTofu, Pulumi, CloudFormation, Helm charts or Kustomize for Kubernetes manifests — and applying them through automation instead of manual console clicks. It makes environments reproducible, reviewable and easy to recreate. → *More:* Ch 1 — Infrastructure as code

Q148. How do you make a Spring Boot service start fast and use less memory in containers? Options: layered JARs and buildpacks for cache-friendly images; Class Data Sharing (CDS) and Project Leyden's AOT cache (ahead-of-time class loading and linking since Java 24, with simpler creation and method profiling added in Java 25) to cut startup; GraalVM native images for millisecond startup and small memory (with build-time restrictions on reflection); virtual threads to serve many requests with few platform threads; and setting `-XX:MaxRAMPercentage` so the JVM respects container memory limits.

Q149. What is a Helm chart? A package of templated Kubernetes manifests with a `values.yaml` file for per-environment settings, installed and upgraded as a versioned "release". It lets you deploy the same service to dev, staging and production with different replica counts, resources and config, and roll back a release.

Q150. What is the Kubernetes Gateway API and how does it relate to Ingress? Ingress is the original, limited API for HTTP routing into a cluster. The Gateway API is its more expressive successor: `Gateway` resources (owned by platform teams) and `HTTPRoute` / `GRPCRoute` resources (owned by app teams) supporting header-based routing, traffic splitting by weight and cross-namespace delegation. Implementations include Envoy Gateway, Istio, NGINX and cloud load balancers.

Q151. How do you roll back a bad deployment? Keep the previous image and config: `kubectl rollout undo`, `helm rollback`, revert the Git commit in GitOps, or switch blue-green back. Automate rollback on canary metric failure. Rollback is only safe if the database migration was backward compatible (expand-and-contract) — which is why you never ship a destructive migration together with the code. → *More:* Ch 1 — Deployment strategies and rollback

---

10. Configuration management

Q152. ⋆ How do you manage configuration across microservices? What is externalised configuration? Keep configuration out of the build artifact so the same image runs in every environment. Sources, by increasing centralisation: environment variables, Kubernetes ConfigMaps and Secrets mounted as files or env vars, a config server (Spring Cloud Config backed by Git), or a store such as Consul, AWS AppConfig or Parameter Store. Secrets go to a dedicated vault. Config should be versioned, auditable and refreshable without redeploying. → *More:* Ch 13 — Configuration Externalization

Q153. ⋆ How does Spring Cloud Config work? A Config Server (`@EnableConfigServer`) serves properties from a backend, usually a Git repository with files like `order-service-prod.yml`. Clients import it on startup with `spring.config.import=optional:configserver:http://config:8888` and receive the properties for their `spring.application.name` and active profiles. Values can be encrypted (`{cipher}…`), and Git gives history and rollback. → *More:* Ch 13 — Worked example: Spring Cloud Config

Q154. ⋆ How do you refresh configuration without restarting a service? Beans annotated `@RefreshScope` (and `@ConfigurationProperties` beans) are rebuilt when a refresh event fires. Trigger it per instance with `POST /actuator/refresh`, or for all instances at once with **Spring Cloud Bus** (`/actuator/busrefresh`) over Kafka or RabbitMQ, often triggered by a Git webhook. On Kubernetes, Spring Cloud Kubernetes can reload on ConfigMap changes, or you roll the Pods. → *More:* Ch 13 — "Refreshing without restarting"

Q155. ⋆ How do you manage secrets in microservices? Never in Git or images. Use a secrets manager — HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager — accessed with short-lived credentials (Kubernetes service account identity), or Kubernetes Secrets encrypted at rest and synced from the vault (External Secrets Operator). Prefer dynamic, short-lived credentials (Vault database

secrets) and rotate regularly. Spring Cloud Vault integrates Vault as a property source. → *More:* Ch 13 — "Security — safeguarding sensitive configuration" · Ch 1 — Encryption, hashing, passwords and secrets

Q156. What if the config server is down? Clients must not fail to start because of it: run the config server with several replicas, cache last-known-good config locally, use `optional:` imports with sensible defaults, or enable fail-fast with retry when config is mandatory. Many Kubernetes shops avoid the extra server entirely by using ConfigMaps, which the platform already keeps highly available. → *More:* Ch 13 — "The configuration server as a single point of failure"

Q157. How do you override Spring Boot properties for different environments? Profiles (`application-prod.yml` activated with `spring.profiles.active=prod`), environment variables (relaxed binding: `SPRING_DATASOURCE_URL` maps to `spring.datasource.url`), command-line arguments, config-tree files mounted from Kubernetes, or the config server. Later sources override earlier ones; command-line arguments and environment variables beat packaged files.

Q158. Config server vs Kubernetes ConfigMaps — which would you choose? ConfigMaps/Secrets: native, no extra server, work for every language, but no built-in history beyond GitOps and refresh needs reload or restart. Spring Cloud Config: Git history, encryption, per-profile layering and bus refresh, but it is another critical service, and it is Spring-specific. On Kubernetes with GitOps, ConfigMaps are usually enough; the config server shines outside Kubernetes or when many Spring apps share layered config. → *More:* Ch 13 — Configuration Externalization

---

11. Security: OAuth2, JWT, mTLS and zero trust

Q159. ⋆ How do you secure microservices? In layers. At the edge: TLS everywhere, an API gateway that authenticates requests (OAuth 2.0 / OpenID Connect tokens), rate limiting and a WAF (web application firewall). Between services: mTLS for encrypted, mutually authenticated traffic and network policies that allow only expected calls. Inside services: validate the token, enforce fine-grained authorization, validate input. Plus secrets in a vault, least-privilege service accounts, image scanning and audit logs. → *More:* Ch 1 — Authentication and authorization · Ch 1 — Least privilege and zero trust

Q160. ⋆ What is OAuth 2.0? What are its roles? OAuth 2.0 is an authorization framework that lets a client obtain a limited-scope access token to call an API on behalf of a user (or itself), without handling the user's password. Roles: **resource owner** (user), **client** (the app), **authorization server** (issues tokens — Keycloak, Okta, Auth0, Spring Authorization Server), **resource server** (the API that validates tokens). OAuth is about authorization; **OpenID Connect** adds an ID token and user info on top for authentication (login). → *More:* Ch 1 — OAuth 2.0, OpenID Connect, SSO and API keys

Q161. ⋆ Which OAuth 2.0 grant types do you use, and when? Authorization code with PKCE for user-facing apps (web, mobile, single-page apps) — PKCE protects the code exchange. **Client credentials** for service-to-service calls with no user. **Refresh token** to get new access tokens without re-login. **Device code** for TVs and CLIs. Do not use the implicit grant (tokens leak through the browser URL) or the resource-owner password grant (the client sees the user's password): the OAuth 2.0 Security Best Current Practice (RFC 9700) says the password grant must not be used and the implicit grant should not be, and the OAuth 2.1 draft drops both.

Q162. ⋆ What is a JWT and how is it validated? A JSON Web Token is a compact, signed token with three Base64URL parts: header (algorithm, key ID), payload (claims such as `sub`, `iss`, `aud`, `exp`, `scope`) and signature. A service validates it locally — without calling the auth server — by checking the signature with the issuer's public key (fetched from its JWKS endpoint and cached, picked by the `kid` header) using only the algorithms you expect (never accept `alg: none`), then `exp` (and `nbf`), `iss` and `aud`. JWTs are signed, not encrypted, so never put secrets in them. In Spring: `spring.security.oauth2.resourceserver.jwt.issuer-uri`. → *More:* Ch 1 — Sessions, cookies, tokens and JWT

Q163. ⋆ How do you revoke a JWT? Opaque tokens vs JWTs? A JWT is valid until it expires, so keep access tokens short-lived (5–15 minutes) and revoke the refresh token; for urgent revocation keep a denylist of token IDs (`jti`) in a shared store such as Redis, checked by the gateway or each resource server, with each entry kept only until that token would have expired anyway. That check brings back a lookup per request, which is the cost of revocability. Opaque tokens are random strings the resource server

must check with the authorization server (token introspection) — instantly revocable but each check costs a call (usually cached). A common mix: opaque at the edge, exchanged by the gateway for a short-lived internal JWT.

Q164. How do you propagate user identity between services? Forward the user's access token on downstream calls (simple, but every service receives a broad token), or use **token exchange** (RFC 8693) to get a narrower token for the downstream audience. For calls with no user, services authenticate as themselves with the client-credentials grant or workload identity. Never trust a plain `X-User-Id` header unless it comes from a component you authenticate (e.g. via mTLS from the gateway).

Q165. ⋆ What is mTLS and why is it used between services? In normal TLS only the server proves its identity with a certificate. In mutual TLS both sides present certificates, so each service knows exactly which service is calling, and traffic is encrypted. Managing certificates for hundreds of services by hand is hard, so service meshes (Istio, Linkerd) issue and rotate short-lived certificates automatically (often with SPIFFE identities) and enforce mTLS in the sidecar without code changes. → *More:* Ch 9 — Sidecar · Ch 1 — HTTPS and TLS

Q166. ⋆ What is zero trust? A security model that assumes the network is already compromised: no request is trusted because it comes from "inside". Every call is authenticated and authorised (mTLS plus tokens), access is least-privilege and short-lived, and traffic is encrypted and logged. It replaces the old "hard shell, soft centre" perimeter model. → *More:* Ch 1 — Least privilege and zero trust

Q167. Authentication vs authorization? Authentication answers "who are you?" (verifying identity with a password, token or certificate). Authorization answers "what are you allowed to do?" (roles, scopes, ownership checks). A request is first authenticated, then authorised for each action. → *More:* Ch 1 — Authentication and authorization

Q168. Where should a single-page app keep its tokens? What is the token-handler (BFF) pattern for security? Tokens in browser storage (`localStorage`) are readable by any XSS script. The recommended pattern is a security BFF: the browser holds only an `HttpOnly`, `Secure`, `SameSite` session cookie, while the BFF performs the OAuth flow, stores tokens server-side and attaches them to API calls. Spring Cloud Gateway with the `TokenRelay` filter is a common way to build it. → *More:* Ch 4 — BFF · Ch 1 — Common web vulnerabilities

Q169. How do you protect services from the OWASP API Top 10 risks? Object-level authorization on every resource (the most common bug: user A reading `/orders/123` of user B), strict input validation, rate limiting, no excessive data in responses, inventory of all API versions, secured management endpoints (Actuator), and protection against server-side request forgery when calling URLs supplied by users. Test these explicitly; gateways alone do not fix object-level authorization.

Q170. What are API keys and when are they acceptable? A static secret string identifying a calling application. They are fine for identifying and rate-limiting partner or server-side clients of low-risk APIs, but they carry no user identity, rarely expire and are easily leaked, so do not use them to authorise user actions. Rotate them, scope them and send them only over TLS. → *More:* Ch 1 — OAuth 2.0, OpenID Connect, SSO and API keys

Q171. How do you secure Kafka or other message brokers? TLS for encryption in transit, authentication of clients (mTLS or SASL/OAUTHBEARER), ACLs so each service can produce or consume only its own topics, encryption at rest, and no sensitive personal data in events unless needed (or encrypt those fields). Remember that events are long-lived copies of data, which matters for privacy rules such as GDPR deletion.

Q172. How do you secure the software supply chain for containers? Use minimal base images, scan images and dependencies for vulnerabilities in CI, generate an SBOM (software bill of materials), sign images (Sigstore cosign) and verify signatures at admission in Kubernetes, pin image digests, run containers as non-root with read-only file systems, and apply Pod Security Standards.

---

12. Testing microservices

Q173. ⋆ How do you test microservices? What types of tests do you use? Follow the test pyramid: many fast **unit tests** of domain logic; **integration tests** of each service with its real infrastructure

(database, broker) using Testcontainers; **component tests** of one service in isolation with its dependencies stubbed (WireMock); **contract tests** between consumers and providers; and a few **end-to-end tests** of critical journeys. Add performance, resilience (chaos) and security tests, and monitor production as a test of last resort.

Q174. ⋆ What is contract testing? What is consumer-driven contract testing? A contract test checks that a provider's API matches what each consumer expects, without deploying both together. In consumer-driven contracts, each consumer writes down the requests it makes and the responses it relies on; the provider's build verifies it satisfies every consumer contract, so a breaking change fails the provider's CI before release. Tools: **Pact** (contracts shared through a Pact Broker, `can-i-deploy` checks) and **Spring Cloud Contract** (contracts generate provider tests and consumer stubs). → *More:* Ch 4 — BFF

Q175. What is Pact? Pact is a consumer-driven contract testing tool available for many languages. The consumer's test runs against a Pact mock server and records a "pact" file of interactions; the pact is published to a broker; the provider replays those interactions against its real API to verify them. The broker's `can-i-deploy` check tells you whether a version is compatible with what is deployed in each environment.

Q176. ⋆ What is the test pyramid (Mike Cohn)? A guideline that a test suite should have many cheap, fast unit tests at the bottom, fewer integration/service tests in the middle, and very few slow, brittle UI/end-to-end tests at the top. In microservices, contract tests replace much of the expensive end-to-end layer.

Q177. Why are end-to-end tests problematic in microservices? They need many services, databases and brokers deployed together in the right versions; they are slow and flaky, failures are hard to pinpoint, and they create a shared environment that becomes a release bottleneck — reintroducing the coupling microservices try to remove. Keep a handful for critical journeys and rely on contracts and component tests for the rest.

Q178. What is Testcontainers and why use it? A library that starts real dependencies — PostgreSQL, Kafka, Redis, even LocalStack for AWS — in throwaway Docker containers during tests. Tests run against the same technology as production instead of in-memory fakes like H2. Spring Boot supports it with `@ServiceConnection`, which wires the container's connection details automatically. Spring Boot 4 uses Testcontainers 2, where each module has its own package and the container classes are no longer generic (`org.testcontainers.postgresql.PostgreSQLContainer` from `testcontainers-postgresql`).

```
@SpringBootTest
@Testcontainers
class OrderRepositoryIT {
  @Container @ServiceConnection
  static PostgreSQLContainer pg = new PostgreSQLContainer("postgres:17");
  // tests now use a real PostgreSQL
}
```

Q179. How do you test a service that calls another service? In unit tests, mock the client interface. In component tests, stub the remote HTTP API with WireMock (or Spring Cloud Contract stub runner) and test timeouts, errors and slow responses too, since those paths are where resilience bugs hide. Verify the stub matches reality with contract tests.

Q180. How do you test asynchronous, event-driven flows? Test producers by asserting the event published (to an embedded or Testcontainers Kafka) for a given command; test consumers by publishing an event and asserting the resulting state, using polling assertions (Awaitility) instead of sleeps. Include tests for duplicate and out-of-order messages, and for poison messages reaching the DLQ. Contract-test event schemas too (Pact supports message pacts).

Q181. What is consumer-side vs provider-side stubbing in Spring Cloud Contract? The provider writes contracts (Groovy/YAML DSL, ideally with consumer input) in its repository; the build generates provider verification tests and publishes WireMock stubs as a JAR. Consumers pull those stubs with the Stub Runner and test against them, so both sides are tested against the same contract.

Q182. What is canary analysis / testing in production? Because staging never matches production exactly, teams verify releases with production traffic under control: canaries with automated metric comparison, feature flags for small user groups, shadow traffic, synthetic monitoring, and chaos

experiments. It complements pre-production tests, it does not replace them. → *More:* Ch 1 — Deployment strategies and rollback

Q183. How do you performance-test a microservice? Load-test with realistic traffic patterns (k6, Gatling, JMeter) against a production-like environment; measure throughput and p95/p99 latency, not averages; find the saturation point and which resource saturates first; test with downstream latency injected; and re-run in CI for key services to catch regressions. → *More:* Ch 1 — Percentiles, tail latency and why averages lie

---

13. Spring Cloud and Spring Boot specifics

Q184. ⋆ What is Spring Cloud? What problems does it solve? Spring Cloud is a set of projects on top of Spring Boot that implement common distributed-system patterns: configuration (Spring Cloud Config, Vault, Kubernetes), service discovery (Eureka, Consul, Kubernetes), client-side load balancing (Spring Cloud LoadBalancer), API gateway (Spring Cloud Gateway), circuit breakers (Spring Cloud Circuit Breaker over Resilience4j), messaging (Spring Cloud Stream, Bus), contract testing (Spring Cloud Contract) and declarative HTTP clients (OpenFeign). It is versioned as a release train: **2025.1 "Oakwood"** is the line for Spring Boot 4.

Q185. ⋆ Spring Boot vs Spring Cloud? Spring Boot builds a single production-ready application quickly: auto-configuration, embedded server, starters, Actuator. Spring Cloud adds the tools for many such applications to work together as a distributed system: discovery, config, gateway, resilience, messaging. Spring Boot is about one service; Spring Cloud is about the system of services.

Q186. ⋆ Why is Spring Boot widely used for microservices? It removes boilerplate with auto-configuration and starters, runs as a self-contained JAR with an embedded server, has production features built in (Actuator health checks, Micrometer metrics and tracing, graceful shutdown, structured logging), integrates natively with Spring Cloud, Kafka, security and Testcontainers, builds container images and native images, and now supports virtual threads with a single property.

Q187. ⋆ What is Spring Cloud Gateway? How do routes, predicates and filters work? Spring Cloud Gateway is Spring's API gateway. A **route** has an ID, a destination URI (which can be `lb://order-service` to load-balance through discovery), **predicates** that decide which requests match (path, host, header, method) and **filters** that change requests or responses (strip prefix, add headers, rate limit, circuit breaker, retry, token relay). In Spring Cloud 2025.x it comes in two flavours — **Server WebFlux** (reactive, Netty) and **Server Web MVC** (servlet, works well with virtual threads) — with the starters `spring-cloud-starter-gateway-server-webflux` and `spring-cloud-starter-gateway-server-webmvc` (the old `spring-cloud-starter-gateway` names were removed in 2025.1). → *More:* Ch 3 — API Gateway

```
@Bean
RouteLocator routes(RouteLocatorBuilder rlb) {
  return rlb.routes()
      .route("orders", r -> r.path("/api/orders/**")
          .filters(f -> f.stripPrefix(1)
              .circuitBreaker(c -> c.setName("orders").setFallbackUri("forward:/fallback")))
          .uri("lb://order-service"))
      .build();
}
```

Q188. What replaced Netflix Zuul, Hystrix and Ribbon? They entered maintenance mode in Spring Cloud Greenwich (2019) and were later removed. Zuul → **Spring Cloud Gateway**; Hystrix → **Resilience4j** (via Spring Cloud Circuit Breaker); Ribbon → **Spring Cloud LoadBalancer**; Hystrix Dashboard/Turbine → **Micrometer** with Prometheus/Grafana. Eureka is still supported. Spring Cloud Sleuth was replaced by **Micrometer Tracing** from Spring Boot 3 onwards.

Q189. ⋆ How do you set up a Eureka server and client? Server: add `spring-cloud-starter-netflix-eureka-server` and annotate the application with `@EnableEurekaServer`. Client: add `spring-cloud-starter-netflix-eureka-client`, set `spring.application.name` and `eureka.client.service-url.defaultZone`; it registers and sends heartbeats automatically (`@EnableDiscoveryClient` is optional). Callers then use the logical name through a `@LoadBalanced` client or `lb://` gateway routes. → *More:* Ch 5 — Service Discovery

Q190. ⋆ What is OpenFeign? What are HTTP interface clients and which should you use now? OpenFeign lets you declare an interface annotated with `@FeignClient("order-service")` and Spring generates the HTTP client, integrated with discovery and load balancing. The Spring Cloud team has declared OpenFeign feature-complete and recommends Spring's own **HTTP interface clients**: an interface with `@GetExchange` / `@PostExchange` methods backed by `RestClient` or `WebClient`. Spring Framework 7 makes them easy to register in groups with `@ImportHttpServices`.

```
public interface InventoryClient {
  @GetExchange("/stock/{sku}")
  Stock stock(@PathVariable String sku);
}
```

Q191. RestTemplate vs RestClient vs WebClient? `RestTemplate` is the old synchronous client: Spring announced its deprecation with Framework 7.0, it is marked deprecated for removal in 7.1, and removal is planned for 8.0. `RestClient` (Spring 6.1+) is the modern synchronous client with a fluent API — the default choice, especially with virtual threads. `WebClient` is the reactive, non-blocking client from WebFlux, for reactive stacks or streaming. All can be `@LoadBalanced` and back HTTP interface clients.

Q192. What is Spring Cloud Circuit Breaker? An abstraction over circuit-breaker libraries (`CircuitBreakerFactory`, `ReactiveCircuitBreakerFactory`) so code is not tied to one implementation; Resilience4j is the main implementation (the Spring Retry implementation is in maintenance-only mode since Spring Cloud 2025.1). You can use it programmatically (`factory.create("inventory").run(supplier, fallback)`) or use Resilience4j's own annotations directly. → *More:* Ch 6 — Circuit Breaker

Q193. ⋆ What is Spring Cloud Stream? A framework for event-driven services that hides the broker behind "binders" (Kafka, RabbitMQ, others). You write plain `java.util.function` beans — `Supplier` (producer), `Function` (processor), `Consumer` (sink) — and bind them to destinations in configuration, e.g. `spring.cloud.function.definition=processOrder` with `spring.cloud.stream.bindings.processOrder-in-0.destination=orders`. It handles serialization, consumer groups, partitioning, retries and DLQs. For Kafka-specific control, plain Spring for Apache Kafka (`@KafkaListener`, `KafkaTemplate`) is also common. → *More:* Ch 11 — Event-Driven

Q194. What is Spring Cloud Bus? It links service instances through a message broker (Kafka or RabbitMQ) so that management events reach all of them, most commonly a configuration refresh: one call to `/actuator/busrefresh` makes every instance reload its `@RefreshScope` beans. → *More:* Ch 13 — Configuration Externalization

Q195. What is Spring Cloud Kubernetes? Integration that lets Spring apps use Kubernetes-native features through Spring abstractions: ConfigMaps and Secrets as property sources (with reload), Kubernetes Services for `DiscoveryClient` and load balancing, so you do not need Eureka or a config server on Kubernetes. Many teams use none of it and rely on plain Kubernetes DNS and mounted config.

Q196. How do you implement a saga or outbox in Spring? Outbox: in a `@Transactional` method, save the entity and an outbox entity; relay with a scheduled poller or Debezium. **Spring Modulith**'s event publication registry gives a ready outbox-like mechanism: `ApplicationEvent`s published in a transaction are stored and re-delivered (and can be externalised to Kafka) if a listener fails. Saga orchestration can be a Spring state machine-style service, or a workflow engine such as Temporal or Camunda with Spring Boot starters; choreography uses Kafka listeners plus compensating handlers. → *More:* Ch 10 — Saga

Q197. ⋆ What is the `@Transactional` pitfall when publishing Kafka messages? Sending to Kafka inside a `@Transactional` database method is a dual write: if the database commit fails after the send, consumers see an event for data that does not exist; if you send after commit and crash, the event is lost. Also, `@Transactional` on a private method or a self-call is ignored because Spring's proxy is bypassed. Use the outbox, or `@TransactionalEventListener(phase = AFTER_COMMIT)` only when losing an occasional event is acceptable. → *More:* Ch 10 — Saga

Q198. How does Spring Boot support Kubernetes probes and graceful shutdown? On Kubernetes, Actuator exposes `/actuator/health/liveness` and `/actuator/health/readiness` automatically, driven by the application's availability state (readiness goes down during shutdown). `server.shutdown=graceful` (the default since Boot 3.4) lets in-flight requests finish within `spring.lifecycle.timeout-per-shutdown-phase` after SIGTERM.

Q199. What is Spring Modulith and when would you use it? A Spring project for building well-structured modular monoliths: each top-level package is an application module; tests verify modules access each other only through public APIs; modules communicate through application events (with a persistent publication registry); and it generates module documentation. It is the recommended way to start before splitting into microservices, and it makes later extraction easier.

Q200. How do you add distributed tracing to a Spring Boot microservice? Add Actuator plus Micrometer Tracing with a bridge (OpenTelemetry) and an exporter (OTLP), or simply Spring Boot 4's `spring-boot-starter-opentelemetry`; set `management.opentelemetry.tracing.export.otlp.endpoint` (Spring Boot 3 called it `management.otlp.tracing.endpoint`) and `management.tracing.sampling.probability`. HTTP server and client calls (`RestClient`, `WebClient`), Kafka, JDBC (with extra instrumentation) and `@Observed` methods produce spans, and trace IDs appear in logs automatically. → *More:* Ch 1 — Observability

---

14. Scenario, troubleshooting and senior-level questions

These are the open-ended questions of senior rounds. Structure every answer the same way: **clarify → find the cause or the constraint → name the pattern(s) → state the trade-off**.

Q201. ⋆ One microservice is slow, but the system is otherwise fine. How do you find the cause? Start from the trace: open a slow request's distributed trace and see which span dominates — its own code, a database query, or a downstream call. Check the service's RED metrics and when the slowdown started (correlate with a deploy, traffic spike or dependency change). Then look at resources: CPU throttling from container limits, GC pauses, thread or connection pool saturation, slow queries (missing index, lock waits), and noisy neighbours. Fix the cause, and add a timeout/bulkhead so this service cannot drag callers down next time. → *More:* Ch 1 — Observability · Ch 1 — Bottlenecks

Q202. ⋆ A downstream service your service depends on is down. What happens and what should you do? Without protection your threads wait on timeouts and you fail too — a cascade. With protection: short timeouts, the circuit breaker opens after the failure threshold and calls fail fast; a fallback serves cached or default data, or the request is queued for later; the bulkhead keeps other features working; alerts fire on the Open state. If the dependency is critical (payments), return a clear error quickly instead of hanging. Long-term, ask whether the call needs to be synchronous at all. → *More:* Ch 6 — Circuit Breaker · Ch 7 — Bulkhead

Q203. ⋆ Design the order placement flow for an e-commerce system with Order, Payment and Inventory services. The Order service saves the order as `PENDING` and an `OrderCreated` event in its outbox, returning 202 Accepted with the order ID. An orchestrator (or choreography) runs the saga: reserve stock in Inventory, then charge the card in Payment (the pivot), then confirm the order. On failure, compensate: release stock, refund, mark the order `CANCELLED`. Every step is idempotent (keyed by order ID), consumers dedupe, the UI shows the pending state and is notified on completion. Add a timeout for stuck sagas and reconciliation with the payment provider. → *More:* Ch 10 — Saga, "Worked example: online shopping"

Q204. ⋆ A customer was charged twice. How could that happen and how do you prevent it? Likely causes: a client or gateway retried a POST after a timeout while the first request had actually succeeded; a message consumer processed a redelivered message twice; or two instances processed the same outbox row. Prevent with an idempotency key on the payment API stored with a unique constraint, idempotent consumers (inbox table), retries in only one layer, `SKIP LOCKED` or a single relay for the outbox, passing the same idempotency key to the payment provider, and a reconciliation job that detects and refunds duplicates. → *More:* Ch 8 — Retry, "Idempotency revisited"

Q205. ⋆ How would you migrate a large monolith to microservices without stopping feature work? Do not rewrite. First modularise inside the monolith and add observability and CI/CD. Put a facade (gateway) in front. Pick a first service with clear boundaries and business value, build it, route its traffic through the facade behind a feature flag, run old and new in parallel and compare results, then switch over and delete the old code. Split data with CDC or events. Repeat capability by capability, measuring deployment frequency and lead time to check it is paying off. → *More:* Ch 2 — Strangler Fig

Q206. ⋆ Kafka consumer lag is growing. What do you check? Lag means consumers process slower than producers write. Check: a sudden traffic spike; slow processing per message (a slow downstream call or

database); a poison message causing endless retries on one partition; frequent consumer-group rebalances (processing taking longer than `max.poll.interval.ms`); too few partitions to add more consumers (parallelism is capped at partitions); or one hot partition from a skewed key. Fixes: speed up or batch the handler, add consumers up to the partition count, increase partitions (which changes key mapping), send poison messages to a DLQ, autoscale on lag (KEDA). → *More:* Ch 1 — Event streaming

Q207. How would you handle a sudden 10× traffic spike (e.g. a flash sale)? Prepare: load-test, pre-scale critical services, warm caches, and set autoscaling on the right metrics. During the spike: rate limit at the gateway, queue orders asynchronously to absorb the burst, shed non-critical work (recommendations, analytics), serve read-heavy pages from CDN and cache, and protect the database (connection pool limits, read replicas). Use a virtual waiting room if even that is not enough. → *More:* Ch 1 — Load shedding and graceful degradation · Ch 7 — Bulkhead

Q208. ⋆ Two services need each other's data and call each other synchronously (a cycle). How do you fix it? A cycle means the boundary is probably wrong or the dependency should be asynchronous. Options: merge the two services if they always change together; move the shared concept into the service that owns it; have one side keep a local copy of the data it needs, updated by events; or introduce a third service for the shared workflow (orchestration). Cyclic synchronous calls also risk deadlocked thread pools during load.

Q209. How do you implement a "get order details" endpoint needing data from five services? Either API composition in a BFF/aggregator — parallel calls with per-call timeouts, returning partial data with flags when optional parts fail — or a CQRS read model (an "order view" service) that subscribes to all five services' events and stores a denormalised document, answering in one fast query at the cost of eventual consistency. Choose composition for low traffic or freshness-critical data; the read model for hot, read-heavy pages. → *More:* Ch 12 — CQRS · Ch 4 — BFF

Q210. A deployment caused errors in production. Walk through your response. Mitigate first, investigate second: roll back (or flip the feature flag off) as soon as the deploy is the likely cause, communicate status, and confirm metrics recover. Then use traces, logs and the diff to find the root cause, write a blameless post-mortem, and add the missing safeguard — a contract test, a canary with automated analysis, an alert. Ask why the canary or tests did not catch it.

Q211. How would you design rate limiting for a public API used by many clients? Identify the client (API key or OAuth client ID); define tiers with limits per second and per day; enforce with a token bucket in Redis at the gateway, using atomic Lua scripts; return 429 with `Retry-After` and rate-limit headers; add per-endpoint limits for expensive operations and a global limit to protect the backend; and consider local in-memory limiters for speed with a shared store for accuracy. → *More:* Ch 1 — Rate limiting and throttling · Ch 3 — API Gateway

Q212. ⋆ A service needs to send an email/notification only after an order is committed. How do you guarantee it is sent exactly once? You cannot guarantee exactly-once delivery to an external email provider, but you can get "at least once, deduplicated". Write an `OrderConfirmed` event to the outbox in the order transaction; a notification service consumes it idempotently (keyed by event ID), sends the email with an idempotency key if the provider supports it, and records it as sent. Retries with backoff handle provider outages; a DLQ catches permanent failures.

Q213. How do you handle a long-running business process (days, e.g. loan approval)? Model it as an orchestrated saga or workflow with durable state: a workflow engine (Temporal, Camunda) or a state machine persisted in the database. Steps are triggered by events or timers, wait for human tasks, have timeouts and escalation, and can resume after crashes. Expose the status through an API and notify users on changes instead of keeping requests open. → *More:* Ch 10 — Saga

Q214. You see intermittent 502/503 errors during deployments. What is the likely cause? Traffic is reaching Pods that are not ready or are shutting down: missing or too-lenient readiness probes, no graceful shutdown, or the load balancer still sending to a Pod after it received SIGTERM. Fix with correct readiness probes, `server.shutdown=graceful`, a short `preStop` sleep so endpoints are removed before the app stops, `maxUnavailable: 0` in the rolling update, and retries for idempotent requests at the gateway. → *More:* Ch 1 — Deployment strategies and rollback

Q215. How do you make a microservice multi-tenant? Choose an isolation model: shared database with a `tenant_id` column on every table (cheapest; enforce with row-level security in PostgreSQL), schema per tenant, or database per tenant (strongest isolation, costliest). Carry the tenant ID in the token,

propagate it in context, apply per-tenant rate limits and quotas so one tenant cannot starve others (a bulkhead per tenant), and include it in logs and metrics.

Q216. ⋆ How do you decide whether a feature should be a new microservice or part of an existing one? Ask: does it belong to an existing bounded context (same language, same data, changes together)? Will a different team own it? Does it need different scaling, availability or technology? Would separating it create chatty calls or a distributed transaction? Default to adding it to the existing service; split when ownership, change rate or scaling needs clearly diverge.

Q217. How would you design a system to guarantee that inventory is never oversold? Make the inventory service the single owner of stock counts and reserve atomically in its database (`UPDATE stock SET available = available - :q WHERE sku = :s AND available >= :q`, checking rows affected). Reserve at checkout with an expiry, confirm after payment, release on cancellation or timeout via the saga. For very hot items, shard counts or use Redis atomic decrements with periodic reconciliation. Don't let the read model (eventually consistent) decide availability. → *More:* Ch 10 — Saga

Q218. How do you reduce the blast radius of failures in a large system? Isolate at every level: bulkheads within services; separate deployments for critical and non-critical paths; per-tenant or per-customer limits; multiple availability zones; cell-based architecture (independent copies of the full stack each serving a subset of users); progressive rollouts per cell or region; and avoiding shared global dependencies that everything calls synchronously. → *More:* Ch 7 — Bulkhead

Q219. What would you look at in a "design a URL shortener/ride-hailing/food delivery with microservices" round? Requirements and scale numbers first; then identify bounded contexts and service list; data ownership and storage per service; synchronous vs event-driven interactions; the critical write path and how it stays consistent (sagas, outbox); read paths and caching; the edge (gateway, auth, rate limits); resilience; observability; deployment. Finish with bottlenecks and trade-offs. Don't create 20 services — justify each split. → *More:* Ch 1 — How to approach a system design interview · Ch 1 — Worked example: URL shortener

Q220. ⋆ What are the most common microservices anti-patterns you have seen? Distributed monolith (lock-step deploys); shared database; too-fine "nanoservices"; chatty synchronous chains; no timeouts; retries at every layer; dual writes without an outbox; business logic in the API gateway; shared domain libraries that force synchronized upgrades; no observability; breaking API changes without versioning; and adopting microservices before having CI/CD and monitoring.

Q221. How do you keep many teams' services consistent without central control? Provide a "paved road" platform: service templates (starter projects with logging, tracing, health checks, security and CI pre-wired), shared base images, an internal developer portal (Backstage) listing owners and APIs, API style guides and linting, and lightweight architecture decision records (ADRs). Standardise interfaces and operations, not internal implementation.

Q222. Your read model (CQRS) is out of sync with the write side. How do you recover? Find why: a consumer bug, lost or skipped events, or out-of-order handling. Fix the bug, then rebuild the projection by replaying events from the beginning (Kafka retention, event store, or a fresh snapshot of the write database via CDC) into a new read store, and switch over when it has caught up. Design projections to be rebuildable from day one and monitor projection lag. → *More:* Ch 12 — CQRS, "Synchronising the two sides"

Q223. How do you deprecate and remove an API endpoint used by other teams? Find the consumers (contract tests, gateway logs, traces); publish a new version and a migration guide; mark the old one deprecated with the `Deprecation` and `Sunset` headers (Spring Framework 7 can add them via its API versioning support); track remaining usage; contact laggards; then remove it once traffic reaches zero.

Q224. When would you choose events over a synchronous call for the same interaction? Events when: the caller does not need the result to answer its own client; several services need to react; the receiver may be down or slow and you do not want to depend on it; or you want to absorb load spikes. Synchronous when: the user needs the answer now (a price check, an authorisation decision), the operation is a query, or strict ordering and immediate consistency outweigh decoupling. → *More:* Ch 11 — Event-Driven · Ch 1 — Synchronous vs asynchronous communication

Q225. ⋆ Tell me about a time a microservices design went wrong. What did you learn? Behavioural, but expect it. Use STAR (Situation, Task, Action, Result) and pick a real example with a technical lesson: a missing timeout that caused a cascade, a dual write that lost events, a split that created a

distributed monolith. Explain the detection, the fix, the safeguard you added (outbox, contract test, alert) and what you would do differently — interviewers want to hear trade-off thinking and ownership.

---

15. Rapid-fire "difference between X and Y" and short-answer round

Phone screens often finish with quick comparisons. Aim for two or three sentences each.

Q226. ⋆ Orchestration vs choreography (in general, not only sagas)? Orchestration: one component directs the others step by step and owns the workflow state — explicit, easy to monitor, but a central dependency. Choreography: each service reacts to events on its own and no one sees the whole flow — loosely coupled, but the end-to-end process is implicit and harder to debug. → *More:* Ch 10 — Saga, "Choosing between them" · Ch 11 — Event-Driven, "Mediator EDA"

Q227. Message queue vs event stream? A queue delivers each message to one consumer and deletes it after acknowledgement — work distribution. A stream (Kafka) keeps an ordered, retained log that many consumer groups read independently at their own offsets and can replay — event history and fan-out. → *More:* Ch 1 — Message queues

Q228. Horizontal vs vertical scaling — which suits microservices? Vertical scaling gives one machine more CPU/RAM; it is simple but has a ceiling and a single point of failure. Horizontal scaling adds more instances behind a load balancer; it suits stateless microservices and gives redundancy, but needs externalised state. → *More:* Ch 1 — Vertical and horizontal scaling

Q229. Container vs virtual machine? A VM virtualises hardware and runs a full guest operating system — strong isolation, heavier, slower to start. A container shares the host kernel and isolates only processes, file system and network with namespaces and cgroups — lightweight and fast, but weaker isolation. → *More:* Ch 1 — Virtual machines, containers and Docker

Q230. Docker vs Kubernetes? Docker builds and runs containers on one machine. Kubernetes orchestrates containers across many machines — scheduling, scaling, self-healing, rolling updates, service discovery. Kubernetes runs images built by Docker (or any OCI tool) using a container runtime such as containerd. → *More:* Ch 1 — Kubernetes basics

Q231. Kubernetes Deployment vs StatefulSet? A Deployment manages interchangeable, stateless Pods with random names that can be replaced freely. A StatefulSet gives each Pod a stable name (`kafka-0`, `kafka-1`), stable storage and ordered start/stop — for databases, brokers and other stateful systems.

Q232. ConfigMap vs Secret? Both inject configuration into Pods as environment variables or files. Secrets are meant for sensitive values: they are kept separately, can be encrypted at rest and access-controlled more tightly — but they are only Base64-encoded by default, so enable encryption and RBAC, or sync them from a vault. → *More:* Ch 13 — Configuration Externalization

Q233. Liveness vs readiness vs startup probe — one line each? Liveness: if this fails, the container is hung, so the kubelet kills and restarts it. Readiness: if this fails, the Pod stays up but is taken out of the Service until it passes again. Startup: holds back the liveness and readiness checks until a slow-booting app has come up. → *More:* Ch 5 — Health checks

Q234. Logs vs metrics vs traces — when do you reach for each? Start with metrics: they are cheap aggregates that drive alerts and dashboards and show that error rate or latency has moved. Then open traces for affected requests to locate the service or call where the time or error appears. Finally read the logs for that trace ID to get the detail — the exception, the input, the decision the code made. → *More:* Ch 1 — Observability

Q235. Circuit breaker vs retry? Retry assumes the failure is brief and tries again; circuit breaker assumes the failure is lasting and stops trying for a while. Used together, the breaker caps the damage retries could do to an unhealthy dependency. → *More:* Ch 6 — Circuit Breaker · Ch 8 — Retry, "Retry vs. fail-fast"

Q236. API Gateway vs service mesh? The gateway handles north-south traffic at the edge — client authentication, public API shape, rate limits per client. The mesh handles east-west traffic between services — mTLS, retries, traffic splitting, telemetry. Many systems use both. → *More:* Ch 3 — API Gateway · Ch 9 — Sidecar

Q237. Event sourcing vs change data capture? Event sourcing makes domain events the source of truth inside the service — intentional, business-meaningful events. CDC derives row-level change events from an ordinary database's log — no code change, but the events mirror table structure, not business intent (so the outbox table is often what CDC reads). → *More:* Ch 12 — CQRS

Q238. Saga vs 2PC — the one-sentence version? 2PC gives atomic, isolated commits across participants at the cost of locks and blocking; a saga gives availability and loose coupling at the cost of isolation and hand-written compensations. → *More:* Ch 10 — Saga

Q239. Stateless JWT vs server-side session? A JWT carries the user's claims and is validated locally by any service — scales easily, but hard to revoke before expiry. A server-side session stores state in a shared store (Redis) referenced by a cookie — easy to revoke, but every service needs access to that store. → *More:* Ch 1 — Sessions, cookies, tokens and JWT

Q240. Strong vs eventual consistency — give a business example for each. Strong: account balances and inventory reservation — a stale read can lose money. Eventual: product reviews, follower counts, search indexes, order history views — a few seconds of staleness is invisible to users. → *More:* Ch 1 — Consistency models

Q241. Spring Cloud Gateway WebFlux vs Web MVC — which one? WebFlux (Netty, reactive) handles very high concurrency with few threads and supports every gateway feature first. Web MVC (servlet) fits teams on the blocking stack; with virtual threads it scales well and is simpler to debug. Pick the one that matches your team's stack. → *More:* Ch 3 — API Gateway

Q242. `@LoadBalanced` — what does it actually do? It marks a `RestClient.Builder`, `RestTemplate` or `WebClient.Builder` bean so Spring Cloud adds an interceptor that treats the host in `http://order-service/...` as a logical service name, resolves it to an instance through the `DiscoveryClient`, and applies the load-balancing algorithm. → *More:* Ch 5 — Service Discovery

Q243. `@RefreshScope` — what is the catch? Refresh-scoped beans are recreated lazily after a refresh event, so references captured elsewhere (e.g. in a constructor of a non-refresh bean) can keep old values, and stateful beans lose their state. Not every property can change at runtime (server port, datasource pool sizes usually need a restart). → *More:* Ch 13 — "Refreshing without restarting"

Q244. Kafka `acks=all` and `min.insync.replicas` — why do they matter? `acks=all` makes the producer wait until all in-sync replicas have the message; `min.insync.replicas=2` (with replication factor 3) means a write fails rather than being accepted by only one broker. Together they prevent losing acknowledged messages when a broker dies — trading a little latency for durability.

Q245. What is a poison message? A message that always fails processing (bad format, unexpected data). Without a limit it blocks its partition forever as the consumer retries it. Detect with a retry count, move it to a DLQ, alert, and keep the partition moving. → *More:* Ch 11 — Event-Driven, "Error handling and message durability"

Q246. Hot partition / hot key — what is it and how do you fix it? One key (a celebrity user, a popular product) receives far more traffic than others, overloading one partition or shard. Fix by salting the key (append a random suffix and aggregate later), splitting the hot entity, caching it, or handling it on a dedicated path. → *More:* Ch 1 — Partitioning and sharding

Q247. Thundering herd vs cache stampede? Both are many clients acting at the same moment. Thundering herd: many waiting clients retry or wake together after an outage. Cache stampede: a hot cache key expires and many requests hit the database at once to rebuild it. Fix with jitter, request coalescing (one rebuild, others wait) and early refresh. → *More:* Ch 8 — Retry, "What is the thundering herd problem?" · Ch 1 — Cache invalidation

Q248. What is the "fallacies of distributed computing" list? Eight false assumptions newcomers make: the network is reliable; latency is zero; bandwidth is infinite; the network is secure; topology doesn't change; there is one administrator; transport cost is zero; the network is homogeneous. Almost every pattern in this book exists because one of them is false. → *More:* Ch 1 — Faults, failures and fault tolerance

Q249. What is service granularity's effect on latency? Quick calculation. If a page needs 6 sequential calls of 20 ms each, that is 120 ms before any work — versus about 20 ms if they run in parallel or the data is local. And availability multiplies: 6 dependencies at 99.9% give about 99.4% combined. Fewer hops and parallel fan-out matter. → *More:* Ch 1 — Availability and the nines

Q250. In one minute: which patterns would you use to build a resilient microservice? Timeouts on every call; retries with exponential backoff and jitter for idempotent operations at one layer; a circuit breaker with fallbacks; bulkheads per dependency; rate limiting and load shedding at the edge; async messaging with outbox, idempotent consumers and a DLQ for state changes; health probes and graceful shutdown; multiple replicas across zones; and observability (RED metrics, traces, SLO alerts) to know when any of it is failing. → *More:* Interview Edition — framework and selection table

---

Sources of the question lists

Question wording and frequency were gathered from these public lists and posts (titles only; all answers above are written for this book):

- InterviewBit — *Microservices Interview Questions* (freshers and experienced lists) — interviewbit.com/microservices-interview-questions
- GeeksforGeeks — *Microservices Interview Questions* — geeksforgeeks.org/java/microservices-interview-questions
- devinterview-io — *microservices-interview-questions* (GitHub) — github.com/devinterview-io/microservices-interview-questions
- DataCamp — *Microservices Interview Questions* — datacamp.com/blog/microservices-interview-questions
- TechPrep — *96 Microservices Interview Questions and Answers (2026)* — techprep.app/blog/microservices-interview-questions
- Adaface — *Microservices interview questions* — adaface.com/blog/microservices-interview-questions
- Coding Shuttle — *Top 20 Microservice interview questions for Advanced Spring Boot Developers* — codingshuttle.com
- Simplilearn — *Microservices Interview Questions* — simplilearn.com/microservices-interview-questions-article
- LambdaTest (DEV Community) — *29 Top Microservice Interview Questions*
- Medium — *Microservices interview questions for Java developers (2026)*; LinkedIn posts — *Microservices scenario-based interview questions* and *Java + Spring Boot + Microservices interview questions (8+ years)*
- LeetCode Discuss — interview-experience posts for backend / Java / Spring roles (topics: saga, circuit breaker, idempotency, Kafka, system design)
- Version facts checked against spring.io release announcements (Spring Cloud 2025.1 "Oakwood", Spring Cloud Greenwich maintenance-mode notice) and Spring Framework 7 / Spring Boot 4 release notes